



Contents lists available at ScienceDirect

Information Processing and Management

journal homepage: www.elsevier.com/locate/ipmSelf-supervised BGP-graph reasoning enhanced complex KBQA via SPARQL generation[☆]Feng Gao^{a,b,c}, Yan Yang^{a,b,c,*}, Peng Gao^d, Ming Gu^c, Shangqing Zhao^c, Yuefeng Chen^e, Hao Yuan^e, Man Lan^{a,b,c}, Aimin Zhou^{a,b,c,*}, Liang He^{a,b,c}^a Lab of Artificial Intelligence for Education, East China Normal University, No. 3663, North Zhongshan Road, Shanghai, 200062, China^b Shanghai Institute for AI Education, East China Normal University, No. 3663, North Zhongshan Road, Shanghai, 200062, China^c School of Computer Science and Technology, East China Normal University, No. 3663, North Zhongshan Road, Shanghai, 200062, China^d School of Cyber Science and Engineering, Qufu Normal University, No. 57, West Jingxuan Road, Qufu, 273165, Shandong, China^e Shanghai Transsion Co., Ltd, No. 36, Xuelin Road, Shanghai, 201203, China

ARTICLE INFO

Keywords:

Knowledge base question answering

Semantic parsing

Intermediate representation

SPARQL query generation

Basic graph pattern

Graph neural networks

ABSTRACT

Knowledge base question answering aims to answer complex questions from large-scale knowledge bases. Although existing generative language models that translate questions into SPARQL queries have achieved promising results, there are still generation errors due to redundancies or errors in the knowledge fed to the generative models and difficulties in representing the implicit logic of knowledge as the specific syntax of SPARQL. To address above issues, we propose TrackerQA, a novel self-supervised reasoning framework based on basic graph patterns (BGP) to determine precise paths and enhance SPARQL generation. First, we develop a contrastive learning semantic matching model to reduce the large knowledge searching space. Then, we build a BGP parser that parses the recalled knowledge and constraints into BGP graphs, which can deconstruct complex knowledge into BGP triples and naturally obtain supervision from gold SPARQL. Next, we design a self-supervised BGP graph neural network that encodes knowledge through graph transformation layers with directed message-passing control and employs a question-aware attention mechanism to predict the exact BGP paths. Finally, a SPARQL generator integrates the paths into a pre-trained language model to improve the performance of SPARQL generation. Experiments on the KQA Pro dataset show that our model achieves state-of-the-art answering accuracy scores of 95.32%, being the closest to the human level at 97.5%, and reasons out KB paths with F1 scores of 0.98 for nodes and 0.99 for edges.

1. Introduction

Knowledge base question answering (KBQA) is a fundamental and challenging task that aims to answer natural language questions over large knowledge bases (KBs), serving as a reliable knowledge resource for chatbots (Zhang, D'Haro, Zhang, Friedrichs and Li, 2023). Recently, semantic parsing-based (SP-based) methods have been utilized to handle complex questions (Lan et al.,

[☆] This work was supported by the Pudong New Area Science and Technology Development Fund, China under Grant No. PKX2021-R05, the Science and Technology Commission of Shanghai Municipality, China under Grant No. 22511105901, and the Shanghai Institute for AI Education, East China Normal University, China under Grant No. 202201003.

* Corresponding authors.

E-mail addresses: fgao@stu.ecnu.edu.cn (F. Gao), yanyang@cs.ecnu.edu.cn (Y. Yang), pgao@qfnu.edu.cn (P. Gao), 51215901012@stu.ecnu.edu.cn (M. Gu), sqzhao@stu.ecnu.edu.cn (S. Zhao), yuefeng.chen@transsion.com (Y. Chen), hao.yuan@transsion.com (H. Yuan), mlan@cs.ecnu.edu.cn (M. Lan), amzhou@cs.ecnu.edu.cn (A. Zhou), lhe@cs.ecnu.edu.cn (L. He).

<https://doi.org/10.1016/j.ipm.2024.103802>

Received 4 February 2024; Received in revised form 28 April 2024; Accepted 30 May 2024

Available online 28 June 2024

0306-4573/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

2023) involving multi-hop reasoning, constrained relationships, and numerical computation. The SP-based KBQA translates natural language questions into executable logical forms, such as SPARQL query (Jiao et al., 2021). The success of pre-trained language models (PLMs) has significantly enhanced the performance of SP-based KBQA (Lewis et al., 2020; Shu et al., 2022; Ye, Yavuz, Hashimoto, Zhou, & Xiong, 2022) in generating logical forms. However, there are still generation errors due to redundancies or errors in the knowledge fed to the generative models and difficulties in representing implicit logic of knowledge as the specific syntax of SPARQL. Therefore, the effectiveness of SPARQL query generation heavily depends on whether or not the proper knowledge and representation required by a question is provided (Shu et al., 2022). As shown in Fig. 1(a), a generative model without considering knowledge (Cao, Shi, Pan et al., 2022) produces inaccurate SPARQL query that overlooks the predicate “replace by” in Fig. 1(b). However, accurately obtaining and representing question-related concepts, entities, and relations in the KB faces three major challenges: (1) a large search space, (2) a gap between natural language question, KB, and SPARQL query, and (3) the lack of a strong reasoning model.

Reducing the large search space is crucial because it comes from a large number of patterns and facts in the KBs and the semantic ambiguity of natural language questions (Saint-Dizier & Moens, 2011). One approach performs iterative reasoning based on anchor entities referenced in the question (Du, Huang, Zhang, & Zhao, 2022; Ye et al., 2022). Another approach generates logical forms from restricted vocabulary and dynamically reasons about n-hop neighborhoods from anchor entities (Gu & Su, 2022; Zhang, Zhang et al., 2023). While these forward search methods can reduce the space, they suffer from irreversible errors and error propagation.

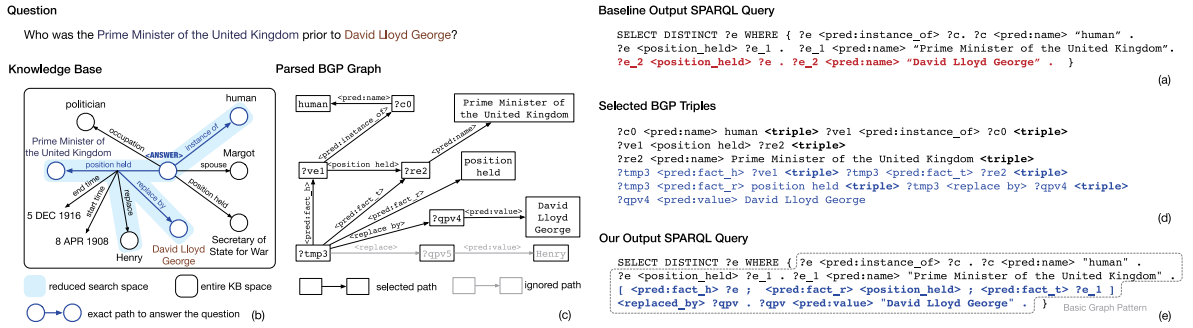
Intermediate representations are often used to bridge gaps between question, KB, and SPARQL query. Some methods utilize dependency parsing (DP) trees (Sun, Zhang, Cheng, & Qu, 2020; Zhang, Zhang, Hui, & Tian, 2022) and abstract meaning representation (AMR) graphs (Kapanipathi et al., 2021; Shivashankar, Benmaarouf, & Steinmetz, 2022) to represent the questions and align them with KBs. These intermediate representations are independent of the KB, which can potentially result in mismatched n-ary parameters and structures, inconsistent granularity (Kapanipathi et al., 2021), and missing long-range dependencies (Sun et al., 2020). Additionally, these representations are incompatible with SPARQL syntax, e.g., missing necessary variables.

To reason out correct knowledge for SPARQL query generation, previous studies (Hu, Wu, Shu, & Qu, 2022; Liu et al., 2022; Ye et al., 2022) utilize unsupervised methods to enumerate and rank candidate knowledge within a few hops from the anchor entity. Others utilized beam search to reason out knowledge paths within a restricted width window (Hu et al., 2022; Zhang, Zhang et al., 2023) and reinforcement learning-based frameworks (Bi et al., 2023) for path selection. Despite their successes, they often employ weak intermediate supervision signals to construct reasoning models, resulting in coarse-grained and erroneous knowledge.

In order to address these issues, we propose a framework called TrackerQA to build a robust self-supervised reasoning model on basic graph pattern (BGP) graphs and reason out exact knowledge in the form of BGP to enhance SPARQL query generation performance. Specifically, we first design a **search space reducer** that employs two semantic matching models inspired by contrastive learning (Lu, Tan, Pergola, Gui, & He, 2022). This reducer globally recalls items and predicates with Top- K semantic similarities to questions in the KBs. Even with a modest value for K , the most relevant items and predicates can be recalled, efficiently reducing the search space and avoiding error propagation in a one-directional search. Next, we innovatively develop a **BGP parser** to convert recalled question-related RDF triples and quadruples into a BGP graph with variables (Fig. 1(c)). Since most forms of SPARQL queries use BGP (Myung, Yeon, & Lee, 2010) (Fig. 1(e)) for graph matching navigation in RDF knowledge bases, BGP is compatible with the RDF knowledge base and SPARQL. In this way, it becomes feasible to naturally extract strongly supervised signals from gold SPARQL queries to label the BGP graphs. Moreover, it helps us simplify RDF quadruples in KB into BGP triples that enable powerful graph neural networks (GNNs) can be applied for reasoning in complex knowledge bases. Following that, we propose an effective self-supervised **BGP-GNN** to reason out exact BGP triples in the parsed BGP graphs. Our approach leverages a PLM to encode knowledge in BGP graphs, incorporates graph transformer layers (Ying et al., 2021) to learn BGP graph representations with directed message-passing control, and employs bidirectional attention to capture correlations between BGP triples and questions. Finally, we feed the selected BGP triples with questions into the **SPARQL query generator** (a BART model) to generate SPARQL with enhanced performance (Fig. 1(e)).

Our main contributions to this field are as follows:

1. We present an effective self-supervised BGP graph reasoning framework for complex KBQA, which utilizes the BGP graph as an intermediate representation to bridge the gap between KBs and SPARQL queries. This framework establishes a robust reasoning model that predicts accurate BGP knowledge to enhance the SPARQL query generations.
2. We propose a BGP parser to parse questions and KB facts into BGP graphs. Our BGP parser simplifies complex KB structures and can obtain strong supervision from gold SPARQL queries, which can facilitate reasoning through employing GNNs and other graph techniques.
3. We design a robust self-supervised BGP-GNN to reason out exact knowledge for SPARQL query generation. BGP-GNN utilizes PLM in knowledge encoding, graph transformer layers with directed message passing, structural encoding between nodes, and question-aware attention mechanisms to model the knowledge and structural representations in BGP graphs, significantly improving path reasoning performance.
4. We evaluate our framework on the popular complex KBQA dataset KQA Pro. Our single and ensemble models achieved answering accuracy scores of 92.4 and 95.32%, respectively, which are new state-of-the-art performances on the leaderboard. These scores are very close to the human level at 97.5%. The ensemble model performed the best in all seven question categories. Our BGP-GNN exhibited notable strengths in knowledge path selections, with F_1 scores of 0.98 and 0.99 for nodes and edges, respectively.



2. Related work

2.1. KBQA

Previous KBQA approaches can be generally categorized into two lines of work: information retrieval-based (IR-based) and semantic parsing-based (SP-based) methods. The IR-based approaches employ semantic matching and similarity measures to extract entities from KBs (Chen, Chang, Chen, Nayak, & Ku, 2019; Chen, Wang, Yu, & Feng, 2022) and rank reasoning paths to obtain to answer entities (Du et al., 2022; Lan, Wang, & Jiang, 2019). In He, Lan, Jiang, Zhao, and Wen (2021), researchers proposed a teacher-student approach to enhance question answering in a student network by leveraging weak supervised signals from answer-to-anchor and anchor-to-answer entities provided by a teacher network. Although IR-based methods have achieved promising results, it is difficult to deal directly with more complex questions, such as counting or verifying answer entities. The SP-based approaches focus on translating natural language questions into structured logical forms (Wang & Qin, 2024). With the success of PLMs, recent SP-based methods used PLM to predict operators (Shen et al., 2020) in logical forms based on question semantics, retrieved subgraphs or subquestions (Das et al., 2022, 2021) for integration into complete logical forms, and ranked the generated form drafts (Liu et al., 2022). For example, Cao, Shi, Yao et al. generated sketch-and-draft forms and then filled arguments into slots. Different from them, we first accurately extract paths from parsed BGP graphs with supervision and then generate complete logical forms through a PLM. Recently, with the emergence of large language models (LLMs), in-context learning models have utilized few-shot (Gu, Deng, & Su, 2023) and zero-shot (Li et al., 2024) settings to translate questions into logical forms in KBQA tasks without fine-tuning. Researchers (Tan et al., 2023) also directly require LLMs to answer questions relied on models' inherent reasoning capabilities and self-knowledge organization without interaction with KBs.

2.2. Intermediate representations in KBQA

SP-based KBQA methods usually utilize intermediate representations to link questions to the KBs. Early studies employed Montague grammar and combinatory categorical grammar to answer simple questions. Ferré (2013) converted simple and patterned questions into Montague grammar and matched their logic to SPARQL queries. To handle the diversity of questions, the DP trees (Sun et al., 2020; Zhang et al., 2022) and AMR graphs (Kapanipathi et al., 2021; Shivashankar et al., 2022) are utilized as intermediate representations. For example, Tran and Nguyen employed a dependency analysis in questions and then applied rules to link them with KB elements. Kapanipathi et al. proposed a neuro-symbolic question answering system to parse questions into AMR graphs and developed a path-based approach to transform the graphs into query graphs aligned with the KB. However, DP tree and AMR graphs still encounter granularity mismatches when interacting with the KB and lack the necessary variables when translating questions into SPARQL queries. In contrast to these methods, we employ a BGP graph as an intermediate representation to bridge the gap between KBs and SPARQL queries, which is seamlessly compatible with the knowledge base and SPARQL. BGP graph also simplifies the complex knowledge base structure and easily obtains strong supervisory signals from gold SPARQL queries.

2.3. GNNs in KBQA

GNNs are suitable for modeling structured semantics in KBs. GNN-based question-answering methods (Zhang, Chen, Fang and Chen, 2023) learn node and edge representations in the non-Euclidean space, allowing them to classify nodes as answer entities in IR-based methods. To tackle edge encoding challenges, Han, Cheng, and Wang introduced the directed hypergraph convolutional

network for updating directed relationship information through hypergraphs and dynamically paying attention to different relations at multiple hops. To overcome the sparsity of KBs, Zhang, Liu et al. proposed the bidirectional recurrent GNN. It modeled the bidirectional information of relations, considering the entity representation in outward and inward directions. Furthermore, Luo et al. introduced a context-path-based GNN to measure relevance and accommodate heterogeneous and schema-rich KBs. This model effectively captured the semantics between nodes and evaluates their significance. Different from the IR-based and mentioned GNN-based methods, we utilize self-supervised signals extracted from gold SPARQL queries and classify BGP triples to form fine-grained paths to generate SPARQL queries, instead of extracting entities as answers. We model the knowledge in the parsed BGP graphs with a PLM and graph transformer layers and reason out the exact BGP triples through question-aware bidirectional attention mechanisms.

3. Research objective

In this article, we aim to construct an effective self-supervised reasoning model on the BGP graph and reason out the exact BGP knowledge paths to generate an accurate SPARQL query for improving the performance of complex KBQA. In detail, our research objectives are as follows:

1. To reduce the search space for complex question answering from the KB, we explore using contrastive learning to develop semantic matching models, recalling question-related items and predicates in the KB.
2. To construct BGP graphs as our intermediate representation, we build a BGP parser that parses questions, interacts with the KB and labels the self-supervised signals from gold SPARQL queries.
3. To establish an effective reasoning model for obtaining exact BGP paths, we propose a practical self-supervised BGP-GNN and explore its powerful capabilities for question-answering path reasoning.
4. To verify the improvement in KBQA performance using reasoned-out BGP paths, we conduct experiments on a complex KBQA dataset. We also perform ablation analysis to evaluate the effectiveness of key components, i.e., search space reduction, BGP graphs, and BGP-GNN.

4. Definition and problem statement

4.1. KBQA

We first define the KB used for the question-answering tasks discussed in this article. Given an RDF KB $\mathcal{K}$, we denote its entities as $\mathcal{E}$, concepts as $\mathcal{C}$, relations as $\mathcal{P}_r$, attributes as $\mathcal{P}_a$, and literal values as $\mathcal{L}$. The union of $\mathcal{E}$ and $\mathcal{C}$ are items represented by $\mathcal{I}$, and the union of $\mathcal{P}_r$ and $\mathcal{P}_a$ are predicates represented by $\mathcal{P}$. In particular, $\mathcal{P}_r$ links to $\mathcal{E}$ and $\mathcal{P}_a$ links to $\mathcal{L}$. Knowledge facts stored in $\mathcal{K}$ comprise triples and quadruples. The set of triples is denoted as $\mathcal{K}_{\text{triple}}$, where $\mathcal{K}_{\text{triple}} \subset \mathcal{I} \times \mathcal{P} \times (\mathcal{I} \cup \mathcal{L})$. The set of quadruples, which are known as qualifier knowledge, is denoted as $\mathcal{K}_{\text{quad}}$, where $\mathcal{K}_{\text{quad}} \subset \mathcal{K}_{\text{triple}} \times \mathcal{P} \times (\mathcal{I} \cup \mathcal{L})$.

Subsequently, we introduce the complex questions we intend to process. A simple question can usually be answered by only retrieving a single triple. However, to answer a complex question, we should accurately retrieve more than one triple and quadruple from $\mathcal{K}$. For example, the simple question “Is David Lloyd George the prime minister of the UK?” can be answered by the triple (David Lloyd George, position held, Prime Minister of the United Kingdom). Nevertheless, to answer the complex question “Who was the Prime Minister of the United Kingdom prior to David Lloyd George?”, we must locate the quadruple (H. H. Asquith, position held, Prime Minister of the United Kingdom, replaced_by, David Lloyd George). In this article, we focus on accurately retrieving triples and quadruples for complex question answering. These retrieved items and predicates, along with the original questions are fed into a generative PLM, enabling to more accurately and flexibly produce correct SPARQL queries with constraints and keywords.

4.2. BGP

A BGP is a set of triple patterns composed of subjects, a predicates, and objects. Assuming that there are four pairwise-disjoint infinite sets, $\mathcal{R}$ symbolizes the internationalized resource identifier to locate items and predicates in the KB, and $\mathcal{B}$, $\mathcal{L}$ and $\mathcal{V}$ denote blank nodes, literals, and variables, respectively. Blank nodes are used as anonymous resources to represent items without a unique identifier. Literals represent values such as strings, numbers, and dates. Variables act as placeholders for resources. Typically, a BGP triple can be expressed as $(s, p, o) \in (\mathcal{R} \cup \mathcal{B} \cup \mathcal{V}) \times (\mathcal{R} \cup \mathcal{V}) \times (\mathcal{R} \cup \mathcal{B} \cup \mathcal{L} \cup \mathcal{V})$. Combining BGP triples in SPARQL queries allows seamless querying of knowledge facts in RDF KBs; RDF facts can be denoted by $(s, p, o) \in (\mathcal{R} \cup \mathcal{B}) \times \mathcal{R} \times (\mathcal{R} \cup \mathcal{B} \cup \mathcal{L})$. As an example, a standard SPARQL query and its BGP are shown in Fig. 1(e): the variable “?e” in BGP triple “?e <pred:instance_of>?c” represents all entities instanced of the concept “?c” (“human”), whereas the “[...]” is a blank node describing some structured resources. The blank node, predicate “replaced_by”, and variable “?qp” constitute a quadruple fact considered as qualifier knowledge. However, the presence of blank nodes means that a quadruple usually cannot apply to graph modeling techniques such as GNNs. To overcome this, we introduce mediator nodes, represented as “?tmp3”, and flatten the quadruples into BGP triples as shown in Fig. 1(c), allowing us to model within GNNs. Hence, the BGP triples are herein defined as $(s, p, o) \in (\mathcal{R} \cup \mathcal{V}) \times (\mathcal{R} \cup \mathcal{V}) \times (\mathcal{R} \cup \mathcal{L} \cup \mathcal{V})$.

4.3. BGP graph

A BGP graph is a directed multigraph denoted as $G = (V, E, f)$ composed of nodes, V , edges, E , and a function $f : E \rightarrow V \times V = \{(v_i, v_j) \mid v_i, v_j \in V\}$. We represent a BGP graph as a set of BGP triples $G = \{(v_i, e_{ij_k}, v_j) \mid v_i, v_j \in N, e_{ij_k} \in E\}$, where k is the index of multiple edges, e.g., (v_i, e_{ij_1}, v_j) and (v_i, e_{ij_2}, v_j) are parallel BGP triples, where e_{ij_1} and e_{ij_2} are multiple edges between v_i and v_j .

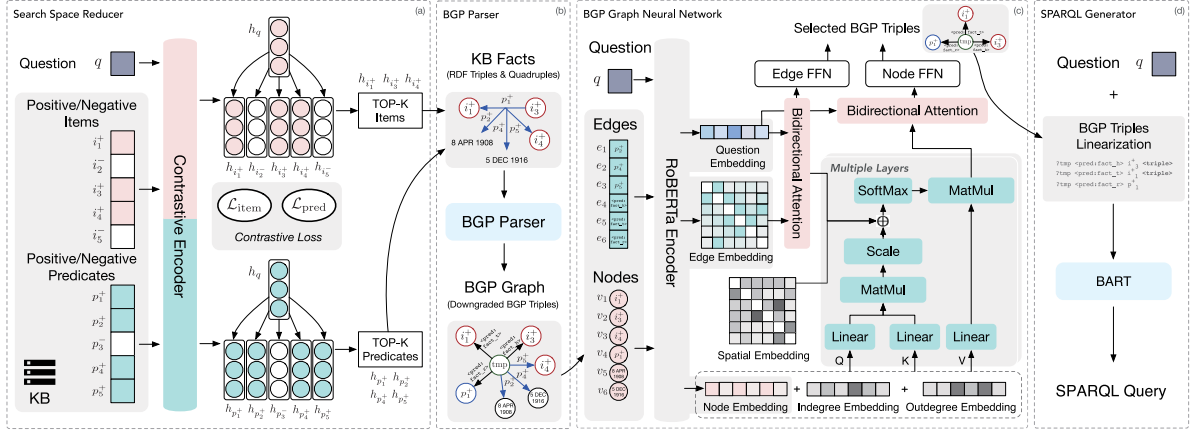


Fig. 2. Overview of proposed framework, TrackerQA. (a) Search space reducer: positive and negative items/predicates are encoded using two contrastive encoders. Encoder parameters are optimized using contrastive loss, obtaining the Top-K items and predicates as output. (b) Process of parsing questions into a BGP graph, involving parsing the question based on recalled items, predicates from KB facts, and the question. (c) BGP-GNN, which takes the BGP graph as input and performs a reasoning process to select relevant nodes and edges and form BGP triples. (d) SPARQL generator, which encodes both the question and linearized BGP triples and produces the SPARQL query.

5. Proposed framework

5.1. Overview

We aim to develop a practical self-supervised BGP graph reasoning framework for complex KBQA, which reason out BGP paths from the KBs on given questions to improve SPARQL query generation performance. As Fig. 2 shows, our TrackerQA consists of four components: search space reducer, BGP parser, BGP-GNN, and SPARQL generator. We first use the search space reducer to retrieve the most relevant knowledge by matching the question with the KBs, and then the BGP parser parses the recalled knowledge into BGP graphs concerning the question. Next, BGP-GNN is responsible for reasoning out the precise BGP paths. Finally, the SPARQL generator combines the paths and the questions to generate SPARQL queries.

5.2. Search space reducer

We use contrastive learning to fine-tune two RoBERTa models (Liu et al., 2019) for semantic matching to retrieve question-related items and predicates. We define items and predicates in the gold SPARQL query as positives, and all others as negatives. Then we match a positive item or predicate with the question as a positive pair, and a negative one as a negative pair. To model the positive and negative pairwise relations between questions and items/predicates, we first generate representations for questions, items, and predicates and group them into positive and negative pairs. Subsequently, we feed these pairs into a training objective for RoBERTa optimization, which pulls positive question-items or question-predicates closer while pushing negative ones further apart. Below, we illustrate item retrieval, which can also be used for predicates.

5.2.1. Data pair construction

Given the entire set of items $\mathcal{I}$ and a collection of questions $\mathcal{Q}$, we generate their representations through the RoBERTa model. Specifically, we encode a question and an item as

$$\begin{aligned} h_q &= \text{avg}(\mathcal{M}(q)), \\ h_i &= \text{avg}(\mathcal{M}(i)), \end{aligned} \quad (1)$$

where q is a question, i is an item, $\mathcal{M}$ is the language model, $\text{avg}(\cdot)$ is the average pooling function, and h_q and h_i are the representations of the question and item, respectively.

We recall the Top-K items by computing their cosine similarities to the question as

$$\mathcal{I}'_q = \text{Top-K}(\cos(h_q, \mathcal{H}_\mathcal{I})), \quad (2)$$

where $\mathcal{H}_\mathcal{I}$ represents the entire set of item representations, $\cos(\cdot, \cdot)$ represents the cosine similarity calculation, $\text{Top-K}(\cdot)$ is the function that obtains the items with Top K similarity scores,¹ and $\mathcal{I}'_q$ represents the recalled items for the question.

¹ We use Faiss (Johnson, Douze, & Jégou, 2021) to accelerate this process.

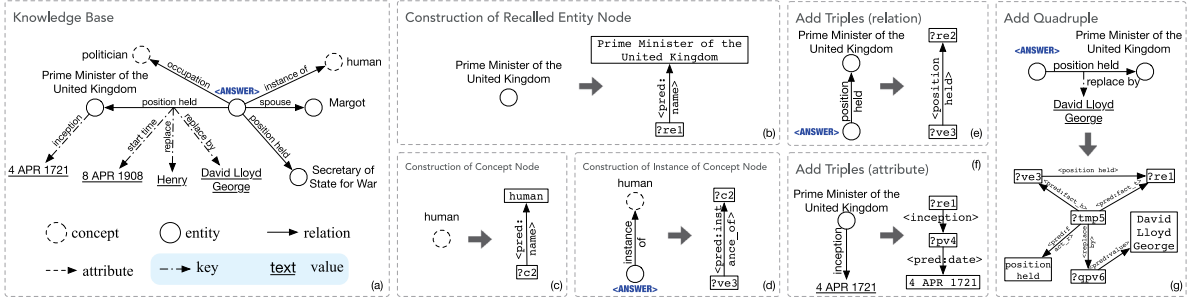


Fig. 3. Operations in the BGP parser. The underlying knowledge base is depicted in (a), wherein the placeholder “<ANSWER>” represents an unknown answer entity for the question in Fig. 1. The parsing steps for recalled entities, concepts, and instances of concepts are shown in (b), (c), and (d), respectively. The RDF triple conversions are divided into relations and attributes, as shown in (e) and (f), while the conversion for quadruples is depicted in (g).

We group the positive and negative pairs of questions and items according to the semantic similarities of representations, obtaining signals from the gold SPARQL query to label recalled items as positive or negative to the question. We define the items in the SPARQL query as positive and denote them as I_q . For each recalled item $i \in I'_q$, if i belongs to I_q , we pair it with q as a positive pair, denoted as (q, i^+) . If i does not exist in I_q , we pair it with q as a negative pair, denoted as (q, i^-) . If i^+ , which exists in I_q , is not recalled in I'_q . We then manually construct the (q, i^+) pair. We maintain a 1:3 ratio between positive and negative pairs.

5.2.2. Contrastive learning for semantic matching

We employ the positive and negative question–item pairs in a training objective to fine-tune $\mathcal{M}$, as given by Eq. (1). We define the contrastive learning loss as

$$\mathcal{L}_{\text{item}} = \log \left(1 + \sum_{(q, i^+) \in I_{\text{pos}_q}, (q, i^-) \in I_{\text{neg}_q}} e^{\tau(\cos(h_q, h_{i^-}) - \cos(h_q, h_{i^+}))} \right), \quad (3)$$

where I_{pos_q} and I_{neg_q} represent the positive and negative data pairs associated with q , respectively, τ is a positive hyperparameter, and $\mathcal{L}_{\text{item}}$ denotes the contrastive learning loss for semantic matching. After completing each training epoch, we perform the data pair construction repeatedly using the updated $\mathcal{M}$.

To recall question-related items in the inference stage, we encode all the items in the KB using $\mathcal{M}$ and store their representations. For a given question q' , we obtain its representation using $\mathcal{M}$ and retrieve its relevant items as

$$\tilde{I}_{q'} = \text{Top-}K(\cos(h_{q'}, H_I)). \quad (4)$$

where H_I is the set of stored item representations, $h_{q'}$ denotes the representation of the given question, and $\tilde{I}_{q'}$ represents the recalled items associated with q' .

5.3. BGP parser

The BGP parser gathers RDF triples and quadruples from the KB related to recalled predicates $\tilde{P}_q$ and items $\tilde{I}_q$ and converts them into BGP triples by using questions to form a graph with heuristic rules as shown in Algorithms 1. Questions play a crucial role in the BGP parser as they determine whether to append BGP triples with specific values or variables to navigate facts in the KB. We classify the recalled items into two categories: entities and concepts. Hereinafter, we denote the recalled entities as $\tilde{\mathcal{R}}\mathcal{E}_q$, the recalled concepts as $\tilde{\mathcal{C}}_q$, and the entities in the KB that are instanced of the recalled concepts as $\mathcal{I}\tilde{\mathcal{E}}_q$. The parsed BGP graph is denoted as G .

5.3.1. Construction of the recalled entities

As shown in Fig. 3(b), for each entity e in $\tilde{\mathcal{R}}\mathcal{E}_q$, we generate a variable name, such as $?re_1$, and add it and e to G . We then draw a directed edge named $\langle \text{pred: name} \rangle$ from $?re_1$ to e . Thus, we parse an entity into a BGP triple $(?re_1, \langle \text{pred: name} \rangle, e)$. All subsequent operations related to e are taken over by $?re_1$. As a key step in search space reduction, we intersect the recalled predicates $\tilde{P}_q$ with all predicates surrounding this entity, resulting in a set of limited predicates, which is denoted as $\tilde{P}_q^e$. We then convert the RDF triples associated with the entity into BGP triples. For each predicate p in $\tilde{P}_q^e$, we retrieve all RDF triples $(e, p, *)$ from the KB and store all objects “*” in the set A .

Next, we determine which RDF triples should be parsed into BGP triples. For each object a in A ,

- If a is an entity, where p is a predicate, and in the recalled entities $\tilde{\mathcal{R}}\mathcal{E}_q$, we obtain the variable node $?re_n$ (recalled entity) or $?ve_n$ (instanced entity) connected to the entity a with a named edge $\langle \text{pred: name} \rangle$ where n is the variable node counter, we then add the BGP triple $(?re_1, p, ?re_n / ?ve_n)$ to G ;

Algorithm 1: BGP Parser

Input: recalled items $\tilde{I}_q$ and predicates $\tilde{P}_q$, natural language question q , and the KB $\mathcal{K}$
Output: a BGP Graph G

```

1 initialize an empty graph  $G$  with multiple directed edges
2 divide  $\tilde{I}_q$  into  $\tilde{\mathcal{E}}_q$  (entities) and  $\tilde{\mathcal{C}}_q$  (concepts)
3  $\tilde{\mathcal{RE}}_q := \tilde{\mathcal{E}}_q$  // assign recalled entities to Recalled Entities
4 id:= 0 // define the node counter
5 foreach  $e \in \tilde{\mathcal{RE}}_q$  do
    // Convert a recalled entity into BGP triples
6    $G, id = \text{ConstructionOfRecalledEntity}(G, e, q, \mathcal{K}, \tilde{\mathcal{E}}_q, \tilde{P}_q, \text{recalled}, id)$ 
7 foreach  $c \in \tilde{\mathcal{C}}_q$  do
    // Convert a concept into BGP triples
8    $G, id = \text{ConstructionOfConcept}(G, c, id)$ 
9 foreach  $c \in \tilde{\mathcal{C}}_q$  do
    // Convert the instances of  $c$  into BGP triples
10   $\tilde{\mathcal{IE}}_q := \text{obtained all Instanced Entities of } c$ 
11  foreach  $e \in \tilde{\mathcal{IE}}_q$  do
12     $G, id = \text{ConstructionOfRecalledEntity}(G, e, q, \mathcal{K}, \tilde{\mathcal{E}}_q, \tilde{P}_q, \text{instanced}, id)$ 
13  $G = \text{AddingRelations}(G, \mathcal{K})$  // Add named and directed edges
14 return  $G$ 

```

• If $\mathbf{a}$ is a literal value, where $\mathbf{p}$ is an attribute, and appears in question q , we add two BGP triples $\{(\text{?re}_1, \mathbf{p}, \text{?pv}_n), (\text{?pv}_n, \langle \text{pred} : \text{value} \rangle / \langle \text{pred} : \text{date} \rangle, \mathbf{a})\}$ to G . As shown in Fig. 3(f), if $\mathbf{a}$ is a specific date, we select the predicate $\langle \text{pred} : \text{date} \rangle$ in the second BGP triple.

Subsequently, we convert the RDF quadruples into BGP triples, allowing us to represent our BGP graphs as GNNs that support the property graphs for qualifier question reasoning. We obtain all quadruples $(e, \mathbf{p}, \mathbf{a}, \{(\mathbf{p}'_*, \mathbf{v}'_*)\})$, where $\{(\mathbf{p}'_*, \mathbf{v}'_*)\}$ is a set of key-value pairs relying on the primary RDF triple. For each $(\mathbf{p}', \mathbf{v}')$ in $\{(\mathbf{p}'_*, \mathbf{v}'_*)\}$, if $\mathbf{p}'$ is in $\tilde{P}_q$, we convert this key-value pair into BGP triples: we create a “?tmp” variable, such as ?tmp₅, which serves as the basis for this conversion. Subsequently, we construct three BGP triples as follows and add them to G :

- $(\text{?tmp}_5, \langle \text{pred} : \text{fact_h} \rangle, \text{?re}_1)$, where ?tmp₅ connects to the head in the primary triple;
- $(\text{?tmp}_5, \langle \text{pred} : \text{fact_t} \rangle, \text{?re}_n / \text{?ve}_n / \text{?pv}_n)$, where ?tmp₅ connects to the tail in the primary triple;
- $(\text{?tmp}_5, \langle \text{pred} : \text{fact_r} \rangle, \mathbf{p})$, where $\mathbf{p}$ is the relation in the primary triple.

Therefore, ?tmp₅ acts as a bridge node to downgrade the RDF quadruple into BGP triples. Subsequently, we generate a “?qp_v” variable, such as ?qp_v₆, to store the key-value pair. We insert $(\text{?tmp}_5, \mathbf{p}', \text{?qp}_v)$ into G as

- if the value $\mathbf{v}'$ appears in q , we add $(\text{?qp}_v, \langle \text{pred} : \text{value} \rangle, \mathbf{v}')$ to G ;
- otherwise, we expect the question to query this value and add $(\text{?qp}_v, \langle \text{pred} : \text{value} \rangle, \text{unknown})$ to G .

After iteration over all elements of $\mathbf{A}$, if no triple can be added to G , we expect the question to query the value in the tail. To achieve this, we add $(\text{?re}_1, \mathbf{p}, \text{?unattr_var})$ to G and scan all the quadruples using the aforementioned process to add them to G .

5.3.2. Construction of the recalled concepts

For each c in $\tilde{\mathcal{C}}_q$, we generate a variable name, such as ?c₂. To parse the concept, we add the BGP triple $(\text{?c}_2, \langle \text{pred} : \text{name} \rangle, c)$ to G , as shown in Fig. 3(c). All the subsequent operations related to this concept are referred to as ?c₂.

5.3.3. Construction of the instances of concepts

For each c in $\tilde{\mathcal{C}}_q$, we obtain instances of c in the KB, denoting them as $\tilde{\mathcal{IE}}_q^c$. Subsequently, we iterate over the instances and follow the same process for constructing the recalled entities. We use a “?ve” variable, such as ?ve₃, to represent the instance and distinguish it from the recalled entity. We do not add BGP triple $(\text{?ve}_3, \langle \text{pred} : \text{name} \rangle, e)$ to the BGP graph. The conversion process of the RDF triples and quadruples for the instances into BGP triples is the same as that for recalled entities. Fig. 3(e) illustrates the conversion of an RDF triple for an instance into a BGP triple, and Fig. 3(g) depicts the conversion of an RDF quadruple for an instance into BGP triples. For all instances of a concept, if no RDF triples in $(\text{?ve}_3, \mathbf{p}, *)$ can be added into G , we add the BGP triple $(\text{?ve}_3, \mathbf{p}, \text{?unattr_var})$ and scan all quadruples to add them to G following the process described in Section 5.3.1.

5.3.4. Adding relations

To enhance the semantic information in the BGP graphs, we enrich the connections between the entity and concept nodes using relations from the KB, including recalled and instanced entities and concepts. We first obtain all variable nodes connected to recalled entities, instances of concepts, and concept nodes. We then create a named directed edge between the obtained variable nodes by verifying that a predicate exists in the KB that connects the entities or concepts they represent.

5.4. BGP-GNN

We propose the novel model BGP-GNN to derive exact BGP triples from the parsed BGP graphs. We initialize graph embeddings, encoding the BGP triples with questions through a PLM. We then introduce the question semantic information into edge embeddings through an attention mechanism and incorporate node centralities into node embeddings. Subsequently, we employ BGP-GNN graph layers to model the BGP graph representations for reasoning, including shortest-path encoding between arbitrary nodes, graph transformer layers, and bidirectional attention mechanism. Finally, we classify the edges and nodes to produce precise and fine-grained BGP triples.

5.4.1. Graph embedding initialization

We initialize node and edge embeddings based on BGP triples, concatenating questions with BGP triples, and input them into a RoBERTa model for contextual encoding. For each BGP triple in G , we encode it combined with a question as

$$X_s = \mathcal{M}([\text{CLS}] \text{ question } [\text{SEP}] v_i e_{ijk} \text{'s is } v_j), \quad (5)$$

where $\mathcal{M}$ is the language model, X_s represents the raw hidden states in the last layer at the token level, and v_i , v_j , and e_{ijk} are the head, tail, and predicate in the BGP triple (v_i, e_{ijk}, v_j) , respectively.

We extract the head, predicate, and tail vectors in a BGP triple by averaging their corresponding tokens in X_s , denoting them as X_s^h , X_s^p , and X_s^t , respectively. For edge embedding, we utilize the predicate vectors as

$$x_{ijk}^{\text{edge}} = X_s^p, \quad (6)$$

where x_{ijk}^{edge} is the edge embedding of edge e_{ijk} .

For node embedding, we consider its appearances as both head and tail entities in all BGP triples of G . For instance, the node embedding of v_i is determined by

$$\begin{aligned} x_i^{\text{head}} &= \frac{1}{|\mathcal{N}_i|} \sum_{v_i \in \mathcal{N}_i} X_{s,i}^h, \\ x_i^{\text{tail}} &= \frac{1}{|\mathcal{O}_i|} \sum_{v_i \in \mathcal{O}_i} X_{s,i}^t, \\ x_i^{\text{node}} &= \text{concat}(x_i^{\text{head}}, x_i^{\text{tail}}), \end{aligned} \quad (7)$$

where $\mathcal{N}_i$ are the BGP triples whose head entity is v_i ; $\mathcal{O}_i$ are the BGP triples whose tail entity is v_i ; x_i^{head} and x_i^{tail} are the averaged head and tail vectors, respectively; $\text{concat}(\cdot)$ denotes the concatenation operation; and x_i^{node} denotes the node embedding of v_i .

Next, we obtain the node embeddings, i.e., $\mathbf{X}^{\text{node}} = \{x_1^{\text{node}}, x_2^{\text{node}}, \dots, x_{|N|}^{\text{node}}\}$, where $|N|$ represents the number of nodes. The edge embeddings are given by $\mathbf{X}^{\text{edge}} = \{x_{12_1}^{\text{edge}}, x_{12_2}^{\text{edge}}, \dots, x_{ijk}^{\text{edge}}\}$.

5.4.2. Question-aware edge representations

We employ a bidirectional attention mechanism to obtain question-aware edge representations. We first calculate the similarity between question and edge embeddings as

$$\mathbf{S} = \text{avg}_{-1} f_a(\text{concat}(\mathbf{X}^{\text{edge}}, \mathbf{X}^{\text{q}}, \mathbf{X}^{\text{edge}} \odot \mathbf{X}^{\text{q}})), \quad (8)$$

where $\mathbf{S}$ is the similarity matrix, $\mathbf{X}^{\text{edge}}$ denotes the edge embeddings, $\mathbf{X}^{\text{q}}$ denotes the raw question embedding in Eq. (5), f_a is a linear transformation, $\odot$ denotes element-wise multiplication, and avg_{-1} is the average pooling in the last dimension.

The bidirectional attention scores are determined as

$$\begin{aligned} a_{eq}^- &= \text{softmax}_{\text{col}}(\mathbf{S}) \odot \mathbf{X}^{\text{q}}, \\ a_{qe}^- &= \text{dup}(\text{softmax}(\max_{\text{col}}(\mathbf{S})))^{\top} \odot \mathbf{X}^{\text{edge}}, \end{aligned} \quad (9)$$

where a_{eq}^- and a_{qe}^- are the edge-to-question and question-to-edge attention scores, respectively, $\text{softmax}_{\text{col}}(\cdot)$ represents the softmax function across the column, $\max_{\text{col}}(\cdot)$ represents the maximum function applied across matrix columns, $\text{dup}(\cdot)$ duplicates “.” for the number of iterations required to match the shape of edges, and “ $\odot$ ” denotes matrix multiplication.

Next, we obtain the question-aware edge representations, $\mathbf{E}$, as

$$\mathbf{E} = \text{concat}(\mathbf{X}^{\text{edge}}, a_{eq}^-, \mathbf{X}^{\text{edge}} \odot a_{eq}^-, \mathbf{X}^{\text{edge}} \odot a_{qe}^-). \quad (10)$$

5.4.3. Node centrality encodings

We integrate node centrality encodings into node embeddings as described in (Ying et al., 2021):

$$h_i^{(0)} = x_i^{\text{node}} + z_{\text{deg}^+(v_i)}^+ + z_{\text{deg}^-(v_i)}^-, \quad (11)$$

Here, $\text{deg}^+(v_i)$ and $\text{deg}^-(v_i)$ denote the outdegree and indegree of node v_i , respectively; z^+ and z^- represent the learnable embedding vectors of the outdegree and indegree, respectively; and $h_i^{(0)}$ denotes the updated node representations.

5.4.4. BGP-GNN graph layers

We first introduce a BGP graph layer with transformers for node modeling. The query-key product similarity matrix A describes the correlations among nodes in the BGP graph. We define A_{ij} as the correlation between v_i and v_j , which is determined as

$$A_{ij} = \frac{(h_i W_Q)(h_j W_K)^\top}{\sqrt{d}} + b_{\phi(v_i, v_j)} + c_{ij}, \quad (12)$$

where h_i and h_j are the node representations of v_i and v_j , respectively; W_Q and W_K are the query and key matrices, respectively; $\phi(v_i, v_j)$ is the shortest-path distance,² which measures the spatial relation between v_i and v_j ; b is a scalar that can learn the value of this distance; and c_{ij} is the accumulated semantic information of edges on the shortest path from v_i to v_j , given as

$$c_{ij} = \frac{1}{N} \sum_{n=1}^N x_n^{\text{edge}} (w_n^E)^\top, \quad (13)$$

where x_n^{edge} is the edge^{nth} representation³ in the shortest path from v_i to v_j , w_n^E is the n th weight embedding matrix for the n th edge, and N is the shortest-path value between these two nodes.

Subsequently, we update the node representation v_i as

$$\text{Attn}(h_i) = \text{softmax}(A_i) h_i W_V, \quad (14)$$

where W_V is a learnable matrix and A_i represents the similarity scores between node v_i and all the other nodes.

We then stack multiple graph layers to enhance the capacity of the BGP-GNN as follows

$$\begin{aligned} \hat{H}^{(l+1)} &= \text{Attn}(\text{Layer-Norm}(H^{(l)})) + H^{(l)}, \\ H^{(l+1)} &= \text{FFN}(\text{Layer-Norm}(\hat{H}^{(l+1)})) + \hat{H}^{(l+1)}, \end{aligned} \quad (15)$$

where $H^{(l)}$ is the node representations at the l th layer, “Layer-Norm” is the layer normalization function, and FFN is a feedforward network.

Similar to the edge representation case, we introduce the question semantic into node representations using a bidirectional attention mechanism:

$$\begin{aligned} S' &= \text{avg}_{-1} f_a(\text{concat}(H, X^q, H \odot X^q)), \\ a_{nq}^- &= \text{softmax}_{\text{col}}(S') \odot X^q, \\ a_{qn}^- &= \text{dup}(\text{softmax}(\max_{\text{col}}(S)))^\top \odot H, \\ H &= \text{concat}(H, a_{nq}^-, H \odot a_{nq}^-, H \odot a_{qn}^-), \end{aligned} \quad (16)$$

where H denotes the node representations updated across all BGP graph layers; S' is the similarity matrix between the nodes and question; a_{nq}^- and a_{qn}^- are the node-to-question and question-to-node attention scores, respectively; and H is the question-aware node representations.

5.4.5. Classification of nodes and edges

We formulate BGP triple reasoning and determination as node and edge classification tasks, assigning labels to BGP triples according to whether they appear in the gold SPARQL queries, which serve as the self-supervised signals for BGP-GNN training. To transform the representations of nodes and edges, we utilize feedforward neural networks that convert them into binary distributions as

$$\begin{aligned} \mathbf{p}^{\text{edge}} &= \text{sigmoid}(\text{FFN}^{\text{edge}}(\mathbf{E})), \\ \mathbf{p}^{\text{node}} &= \text{sigmoid}(\text{FFN}^{\text{node}}(\mathbf{H})), \end{aligned} \quad (17)$$

where FFN^{edge} and FFN^{node} represent two feedforward networks, and $\mathbf{p}^{\text{edge}}$ and $\mathbf{p}^{\text{node}}$ are the distributions.

We optimize the parameters in the BGP-GNN using the binary cross-entropy loss as

$$\begin{aligned} \mathcal{L}_{\text{BGP}} &= \mathcal{L}_{\text{node}} + \mathcal{L}_{\text{edge}} \\ &= - \sum_{i=1}^{|N|} [y_i^{\text{node}} \log(p_i^{\text{node}}) + (1 - y_i^{\text{node}}) \log(1 - p_i^{\text{node}})] \\ &\quad - \sum_{j=1}^{|E|} [y_j^{\text{edge}} \log(p_j^{\text{edge}}) + (1 - y_j^{\text{edge}}) \log(1 - p_j^{\text{edge}})], \end{aligned} \quad (18)$$

where y^{node} and y^{edge} are the labels assigned to nodes and edges, respectively, in the BGP triples; $\mathcal{L}_{\text{node}}$ and $\mathcal{L}_{\text{edge}}$ represent the losses of the predicted and real distributions for nodes and edges, respectively; and $\mathcal{L}_{\text{BGP}}$ denotes the BGP-GNN loss.

² When two nodes are not reachable, we set the distance to -1 .

³ When there are parallel edges between two nodes, we randomly select one.

5.4.6. Selection of BGP triples

To extract precise paths from the BGP graph, we first create an empty set, $\mathcal{T}$, to store the selected BGP triples. Then, we select two nodes from the preserved nodes in the binary classification, v_i and v_j , and we check whether they are connected by any one-hop connection in the BGP graph. If so, we add the BGP triple to $\mathcal{T}$ according to the following two rules:

- if the two nodes are connected by only one edge, we directly add the BGP triple (v_i, e_{ij}, v_j) to $\mathcal{T}$;
- if there are multiple edges between the two nodes, we select the BGP triples according to the preserved edges in the classification. For example, if an edge set $e_{ij} = \{e_{ij1}, e_{ij2}, e_{ij3}\}$ exists between the two nodes, and both e_{ij2} and e_{ij3} are preserved in edge classifications, we add the two BGP triples (v_i, e_{ij2}, v_j) and (v_i, e_{ij3}, v_j) to $\mathcal{T}$.

5.5. SPARQL generator

We employ a sequence-to-sequence generative PLM to produce BGP graph-enhanced SPARQL queries. Further, we linearize selected BGP triples as sequential paths and generate SPARQL queries along with questions.

5.5.1. BGP triple linearization

We select a BGP triple from $\mathcal{T}$ and prioritize the BGP triples containing concept variables, entity variables, and others. We employ a depth-first traversal, starting from the tail entity in the selected BGP triple and proceeding to the other BGP triple. When the tail entity in the selected BGP triple is a termination, we select another BGP triple from $\mathcal{T}$ to perform depth-first traversal until we include all BGP triples in the linearization. We obtain a text sequence $\mathbf{t}$ following the order of BGP triple linearization wherein the triples are separated with a special token “<triples>”.

5.5.2. SPARQL query generation

We utilize BART (Lewis et al., 2020) to generate SPARQL queries with reasoned BGP triples. We concatenate the BGP triple linearization with the question as $\mathbf{s} = q \text{ [SEP] } \mathbf{t}$, where $\mathbf{s}$ is the concatenated sequence to be encoded, fine-tune the model autoregressively with the cross-entropy loss, and generate the SPARQL query sequence as follows:

$$p(\mathbf{o} \mid \mathbf{s}) = \prod_{t=1}^{|\mathbf{o}|} p(y_t \mid y_{<t}, \mathbf{s}), \quad (19)$$

where $\mathbf{o}$ represents the generated SPARQL query token-by-token, $p(\mathbf{o} \mid \mathbf{s})$ is the conditional probability of generating query $\mathbf{o}$ given the input sequence $\mathbf{s}$, $|\mathbf{o}|$ is the length of the query, and $p(y_t \mid y_{<t}, \mathbf{s})$ is the probability of generating each token of the query, given the previous tokens ($y_{<t}$) and the input sequence $\mathbf{s}$.

6. Experiment and analysis

6.1. Dataset description

To evaluate our approach for complex question-answering tasks, we explored several well-regarded KBs and selected Wikidata, a free, collaborative, and large-scale KB distinguished by its active community and frequent updates, which facilitates easy access to and reuse of structured data. For the question-answering tasks, we chose a challenging complex KBQA benchmark, **KQA Pro** (Cao, Shi, Pan et al., 2022), because it is designed with a variety of question types to evaluate the approach’s robustness, including multi-hop, qualifier, comparison, logical, count, verify, and zero-shot questions. Additionally, the large scale of the benchmark, comprising 117,970 questions, and the blind testing on the test dataset ensure a fair comparison to the other baselines. Specifically, the KQA Pro dataset, designed to assess KB knowledge reasoning capabilities, encompasses an extensive collection of knowledge facts, including at least 589,873 RDF triples and 309,407 quadruples. In addition, it features seven types of complex questions, which serve to evaluate the effectiveness in generating diverse SPARQL queries. The dataset’s 117,970 questions were randomly divided into three sets at an 8:1:1 ratio by the dataset creator. The training, validation, and SPARQL queries unseen test sets comprised 94,376, 11,797, and 11,797 questions, respectively. For the question types, the multi-hop questions require entity reasoning among different numbers of hops. The qualifier questions involve quadruple facts, whereas the comparison questions require comparing sets of entities. Logical questions entail the application of logical union or intersection operations. Count questions seek the number of answer entities, and verify questions require binary “yes” or “no” responses. Zero-shot questions involve answers not encountered in the training set.

We use the accuracy score on the test set as the metric to evaluate model performance, which is the percentage of correctly answered results out of all questions in the set. Due to the blind test settings, we submit the results executed by the generated SPARQL queries to the dataset creators and obtain the accuracy score from their official evaluation.

6.2. Implementation details

To facilitate the reproducibility of our experimental outcomes, we present crucial configurations, including the hyperparameters, hardware settings, software environments, and details in training stages.

- **Search space reducer.** We stored H_L^L and H_P^L in two Faiss libraries to retrieve the Top-5 items and Top-20⁴ predicates. The models were implemented using PyTorch v1.12.1 and trained on a single NVIDIA GeForce RTX 3090 GPU with 24 GB of memory. We used a batch size of 24, set the hyperparameter τ to 20, the learning rate to 3e-5, and employed the AdamW optimizer. After each training epoch, we updated H_L^{L+1} and H_P^{L+1} in the libraries using the RoBERTa models trained in the L th epoch. The maximum number of training epochs was set to 20, and we employed an early-stop strategy in which the training process halted if the validation loss did not decrease for five epochs.
- **BGP-GNN.** We implemented our BGP-GNN using the Graphormer code⁵ based on PyTorch v1.12.1 and trained on a single NVIDIA A100 GPU with 40 GB of memory. We set the maximum shortest path distance to five and the maximum number of nodes in the BGP graph to 128. To initialize graph embeddings, we employed a RoBERTa-base model. We stacked 12 BGP graph transformer layers in our BGP-GNN, adopted a batch size of six, and utilized the AdamW optimizer with a learning rate of 3e-5. We conducted training for a maximum of 25 epochs, employing an early-stop strategy in which the training was halted if the node F_1 score did not increase within ten epochs.
- **SPARQL generator.** We employed the BART-base model from HuggingFace⁶ as our SPARQL generator. Our implementation, based on PyTorch v1.12.1, was trained on two NVIDIA GeForce RTX 3090 GPUs, each with 24 GB of memory. For training, we utilized a batch size of 12 and set the learning rate to 3e-5 with the AdamW optimizer. The maximum number of training epochs was 15, and we incorporated an early-stop strategy in which the training was halted if the validation loss did not decrease within six epochs.

6.3. Baselines

To ensure a comprehensive and fair comparison, we selected baselines that are representative for KBQA and have been officially evaluated by the KQA Pro benchmark. Therefore, the baselines for our comparison are as follows.

- **SRN** (Qiu, Wang, Jin, & Zhang, 2020) is a deep reinforcement learning model that formulates multi-relation question answering as sequential decision problems in KBs, treating the final answers as weak supervision and learning the optimal sequential decision-making process for answering questions.
- **KVMemNet** (Miller et al., 2016) organizes facts from the KB into a memory structure composed of key-value pairs. It iteratively reads the memory to update its query vector. After a fixed number of hops H , the model predicts answers from a pool of possible candidates.
- **EmbedKGQA** (Saxena, Tripathi, & Talukdar, 2020) learns entity embeddings in the KB and utilizes them to match question semantics, scoring each (head entity, question) pair against all entities in the bases. The entity with the highest score is selected to answer the question.
- **RGCN** (Schlichtkrull et al., 2018) updates the entity embeddings in the KB using a relational GNN. This approach allows the learning of representations for predicates in a graph and selects the entity that serves as the answer to the question.
- **RNN** (Dong & Lapata, 2016) with an attention mechanism is trained from scratch in a sequence-to-sequence paradigm. This approach entails encoding natural language questions and decoding their SPARQL queries.
- **BART** (Lewis et al., 2020) is fine-tuned using a sequence-to-sequence paradigm that encodes natural language questions and decodes their corresponding SPARQL queries through supervised learning.
- **CFQ IR** (Herzig et al., 2021) rewrites the triples in SPARQL queries to a reversible intermediate representation that better corresponds with natural language. It produces the representations from the questions through a BART model and translates them into SPARQL queries through rules without interacting with the KB.
- **GraphQ IR** (Nie et al., 2022) utilizes GraphQ as an intermediate representation to facilitate the generation and compilation of SPARQL queries. It employs a BART model to convert natural language questions into specifically defined representations, which are then compiled into SPARQL queries using an abstract syntax tree without interacting with the KB.
- **ChatGPT** and **GPT-4** (Tan et al., 2023), two LLMs in the GPT family, are required to answer questions from the test set using their inherent reasoning capabilities, without accessing the KB.
- **FlexKBQA** (Li et al., 2024) samples SPARQL queries from KBs using templates and employs a LLM to translate these queries into natural language questions. Then, FlexKBQA trains a lightweight BART model to explore the performance of LLMs in KBQA tasks under a zero-shot setting.
- **GRC Model** and **GRC MTL** are two ensemble models on the leaderboard whose details have not been disclosed.
- **TrackerQA*** is our ensemble model, which preserves generated SPARQL queries in three different trained epochs of the BART model, executes them on the KB endpoint, and uses a majority voting strategy to determine the final answer to each question.

To sum up, these baselines can be categorized into four groups: (1) IR-based methods, such as SRN, KVMemNet, EmbedKGQA, and RGCN; (2) SP-based methods, including RNN, BART, CFQ IR, and GraphQ IR; (3) LLM-incorporated methods, like ChatGPT, GPT-4, and FlexKBQA; and (4) ensemble methods, such as the GRC Model and GRC MTL. We compare our approach to these baselines and analyze our performance across these groups.

⁴ We determined the values of K by balancing retrieval effectiveness and computing efficiency.

⁵ <https://github.com/TencentYouTuResearch/HIG-GraphClassification>.

⁶ <https://huggingface.co/facebook/bart-base>.

Table 1

Comparison of accuracy scores (%) of overall experimental results with baselines.

Single models	Overall	Multi-hop	Qualifier	Comparison	Logical	Count	Verify	Zero-shot
<i>Human</i> (2022)	97.50	97.24	95.65	100.00	98.18	83.33	95.24	100.00
SRN (2020)	–	12.33	–	–	–	–	–	–
KVMemNet (2016)	16.61	16.50	18.47	1.17	14.99	27.31	54.70	0.06
EmbedKGQA (2020)	28.36	26.41	25.20	11.93	23.95	32.88	61.05	0.06
Blind GRU* (2020)	34.36	33.25	28.82	25.77	34.17	39.43	62.15	0.06
RGCN (2018)	35.07	34.00	27.61	30.03	35.85	41.91	65.88	0
RNN (2016)	41.98	36.01	19.04	66.98	37.74	50.26	58.84	26.08
BART (2020)	89.68	88.49	83.09	96.12	88.67	85.78	92.33	87.88
CFQ IR (2021)	88.96	87.51	81.32	95.70	90.33	86.23	92.20	87.12
GraphQ IR (2022)	91.70	90.38	84.90	97.15	92.64	89.39	94.20	94.20
ChatGPT (2023)	47.93	–	–	–	–	–	–	–
GPT-4 (2023)	57.20	–	–	–	–	–	–	–
FlexKBQA (2024)	46.83	–	–	–	–	–	–	–
TrackerQA (Ours)	92.40	91.29	88.69	97.24	90.65	87.81	94.27	92.01
Ensemble models								
GRC model* (2022)	91.94	90.78	88.94	94.76	91.13	88.56	90.26	87.37
GRC MTL* (2022)	93.85	92.87	92.13	96.68	93.71	89.32	91.09	93.85
TrackerQA* (Ours)	95.32	94.76	93.83	98.08	94.66	89.47	94.54	96.64
Δ	+2.92	+3.47	+5.14	+0.84	+4.01	+1.66	+0.27	+4.63

1. We executed generated SPARQL queries to obtain answers for the test set and submitted them to the dataset developers for blind evaluation, ensuring a fair comparison with the baselines. The highest score is presented in **bold**.

2. Unavailable performance, undisclosed baseline, and improvement in the ensemble relative to single model are given by “–”, “*”, and “ Δ ”, respectively.

6.4. Experimental results

We present the overall performance of the proposed and baseline models in Table 1. The results indicated that TrackerQA significantly outperformed the baselines. The results are presented below.

Overall, TrackerQA outperformed the SRN, KVMemNet, EmbedKGQA, Blind GRU, and RGCN models. The baselines are IR-based models that tightly incorporate facts into the KB and learn entity embeddings for answering questions but have difficulty dealing with complex questions. For example, they cannot answer zero-shot questions owing to the lack of unseen entity embeddings. For the count and verify questions, they lack explicitly designed mechanisms to count and distinguish eligible entities. Our framework does not rely on embeddings for specific entities but rather understands the semantic relationships between questions and the KB, generating SPARQL queries that can handle the count and verify questions using the “COUNT” aggregation function and “ASK” query form, respectively. In addition, SRN reinforcement learning only models entity–relation–entity in multi-hop questions, not entity–attribute–literal and qualifier questions. We address this issue by adding these facts to the BGP graph and selecting gold facts through BGP-GNN reasoning.

TrackerQA outperformed the RNN, BART, CFG IR, and GraphQ IR models, which generate SPARQL queries via SP-based methods. We observed that the generative PLMs are crucial to SPARQL query generation. The RNN model achieved only 41.98% accuracy, significantly lower than that of the BART model (89.69%). Although the CFQ IR and GraphQ IR models employ intermediate representations to connect natural language with SPARQL queries, their lack of fine-grained knowledge path retrieving from the KB resulted in inferior performance than our model. They miss unified resource identifiers for the concepts, entities, and predicates, making their SPARQL queries unfaithful to knowledge facts and leading to incorrect answers, whereas TrackerQA provides the exact BGP paths, including this information through parsing the questions into BGP graphs and reasoning with our BGP-GNN.

For the LLM-incorporated methods, focusing on inherent reasoning capabilities, the accuracy scores were 47.93% for ChatGPT and 57.20% for GPT-4. These results indicate significant space for improvement in reasoning abilities compared to fully-supervised methods. Additionally, these methods lack interpretability when answer the questions, compared to SP-based methods. Although FlexKBQA achieved an accuracy score of only 46.83%, it represents a pioneering effort in exploring zero-shot KBQA. The method specifically detected the KB schema using synthetic data and utilized questions from the KQA Pro to reduce the distribution shift between synthetic and real user queries. Our approach benefits from self-supervised signals from the gold SPARQL queries, which not only provide interpretability in an SP-based manner but also allow direct interaction with the KB, thereby mitigating the distribution shift issue.

For the specific question type, our model exhibited satisfactory performance. The single model achieved the best performance for the qualifier, multi-hop, comparison, and verify questions, outperforming the GraphQ IR model by 3.79, 0.91, 0.09, and 0.07%, respectively. The multi-hop question performance illustrates our model’s stronger multi-hop reasoning ability because the BGP-GNN can reason out exact paths between multi-hop entities, aiding reasonable, accurate SPARQL query generation. The improved performance on the qualifier questions relative to the CFQ and GraphQ intermediate representations indicates the BGP graph’s excellent parsing ability; it fairly and effectively downgrades RDF quadruples to BGP triples for graph-based reasoning. We observed that the proposed single model was comparable to GraphQ IR when answering logical, count, and zero-shot questions that involve

Table 2
Experimental results of ablation study.

Settings	Acc (%)	Δ (%)
<i>TrackerQA</i>	<i>92.46</i>	–
Vanilla BART	88.45	–4.01
Repl. RoBERTa with ELQ	89.81	–2.65
w/o BGP Graph	89.15	–3.31
w/o BGP-GNN reasoning	88.89	–3.57
w/o BGP Graph and with AMR graph	88.88	–3.58
Repl. BART with transformer	75.98	–16.48
Repl. DFS with shuffled in generation	92.07	–0.39

1. “w/o”, “with”, and “Repl.” denote, respectively, removal of a specific mechanism, utilization of the mechanism, and replacing one mechanism with another.
2. The difference in performance relative to the entire workflow, is represented by “ Δ ”.

complex SPARQL query skeletons. This is due to limitations in the BART model used in the SPARQL generator, which can be mitigated by using ensemble SPARQL queries generated by multiple BART models.

We further investigated the performance of TrackerQA in ensemble settings, executing SPARQL queries generated from various trained epoch BART models in the SPARQL generator. The results indicated that our approach surpassed other ensemble models significantly, with performance close to that of humans. Our ensemble model outperformed the two undisclosed models in answer accuracy by 3.38 and 1.47%, respectively, and outperformed the other two ensemble models across the leaderboard for the specific question type. We believe that our parsed BGP graphs and the classification results of the BGP-GNN are of high quality. However, the BART model limits the application of these exact and fine-grained paths, which can be boosted and stably improved using the ensemble of generated SPARQL queries. We will analyze the errors generated by the BART model in Section 6.7.

6.5. Ablation experiments

To explore the contributions of the components, we evaluated the framework under different settings on the validation set. The ablation study results are listed in Table 2.

- **Vanilla BART.** We fine-tuned a pre-trained BART model to translate natural language questions into SPARQL queries directly. This model retained only our SPARQL generator, enabling evaluation of the performance of the model without interacting questions with the KB.
- **Repl. contrastive learned RoBERTa with ELQ.** We replaced the search space reducer with the ELQ toolkit (Li, Min, Iyer, Mehdad, & Yih, 2020), an end-to-end entity recognition toolkit. This variant assesses the contribution of two contrastive learned language models for semantic matching.
- **w/o BGP Graph.** We removed the BGP graphs to observe the advantages of our proposed intermediate representation. We directly concatenated the question with the recalled items and predicates and fine-tuned the BART model to generate SPARQL queries. This model is similar to that presented in Hu et al. (2022) and Ye et al. (2022), where the interacting process, unlike our graph-style approach, was performed linearly.
- **w/o BGP-GNN Reasoning.** In this version, we disabled the reasoning capabilities of the BGP-GNN, instead of linearizing the parsed BGP graph (containing coarse-grained and noisy paths) and using the subsequent components to evaluate the contribution of selectivity in the BGP-GNN.
- **w/o BGP Graph and with AMR Graph.** We incorporated AMR graphs for question parsing while turning off the BGP graph to observe the impact of different intermediate representations. We employed a high-quality AMR parser (Bai, Chen, & Zhang, 2022) to parse questions into AMR graphs aligned with the entities in Wikidata and performed depth-first traversal linearization on the graph to generate text sequences concatenated with the questions. The BART model was fine-tuned to generate the SPARQL.
- **Repl. BART with Transformer.** To explore the benefits of employing a pre-trained language model for SPARQL query generation, we implemented a variant where the BART model was replaced with a 12-layer encoder and decoder transformer architecture as the generator. This variant initialized model parameters randomly and trained the generator from scratch.
- **Repl. DFS with Shuffled in Generation.** In linearizing the BGP graph in the generator, we modified it by replacing the depth-first traversal strategy with shuffled ordering to disrupt the well-organized order of BGP triples within the graphs.

6.5.1. Impact of search space reducer through contrastive learning

Replacing the search space reducer with ELQ (Li et al., 2020) decreased the accuracy by 2.65% to 89.81%. This is attributed to two factors: (1) The ELQ toolkit has performance limitations, which makes it hard to identify all the items mentioned in the question. (2) The ELQ toolkit does not extract the predicates in the question and, therefore, cannot successfully construct a complete BGP graph from the question. Our contrastive learning approach was demonstrated to be advantageous for identifying items and predicates to build an efficient reduced search space.

6.5.2. Effectiveness of BGP graph as intermediate representation

The variant without the BGP graph but with the AMR graph achieved only 88.88% accuracy. Despite the AMR graph's ability to align with entities in Wikidata, its failure to align with predicates resulted in insufficient information for generating SPARQL queries. We provide both the entities and predicates in SPARQL query generations. Furthermore, similar to Hu et al. (2022) and Ye et al. (2022), when we removed the BGP graph from the framework and directly concatenated the recalled items and predicates to the questions, the accuracy score decreased to 89.15%, a 3.31% reduction compared with the complete framework and a mere 0.7% improvement over the vanilla BART. Because of the high accuracy in recalling items and predicates (see Section 7.1), we reckon that offering structural information and sharing a similar syntax to the SPARQL query in BGP graph is also crucial to SPARQL query generation. Simply concatenating the recalled items and predicates with questions would require BART to reconstruct triples and quadruples, which differs from the pre-training tasks in BART.

6.5.3. Effectiveness of BGP-GNN reasoning

Providing the full BGP graph to the SPARQL generator is even less effective than not providing any information. The main reason is that a large amount of noisy knowledge information seriously hurts the performance of the generative model, which is why we reason the exact knowledge paths. Additionally, the model's input context is limited, and the scale of the knowledge base is often so large that, without careful filtering, it is sometimes impractical to directly input it into the model.

6.5.4. Evaluation of the SPARQL query generator variants

Pre-training the language model has a significant impact on the quality of query generation. When the SPARQL generator is trained from scratch, the accuracy is only 75.98%, which corresponds to a reduction of 16.48%. During SPARQL generation, the questions should be with KB knowledge since the vanilla BART has an accuracy of only 88.45%, a decrease of 4.01%, and is the lowest scoring of the variants using the PLM. When using random order instead of depth-first traversal during the linearization of the BGP triples, the accuracy is reduced by 0.39%, suggesting that a well-formatted encoding order improves the quality of the generated SPARQL queries.

6.6. Case study

To understand the improvements in SPARQL query generation, we compared the SPARQL queries produced by our model and the BART models in Table 3, using samples from the validation dataset. We also present the BGP triples selected through the BGP-GNN.

The first two cases demonstrate the effectiveness of our BGP triples for item and predicate retrieval, respectively. Whereas the BART model directly copied a celebrity's nickname, e.g., *Charlie Laughton*, from the question, our approach performed item retrieve, which aligned the nickname with the official name of *Charles Laughton* in the KB. This allowed us to generate an accurate SPARQL query. In the second case, the BART model fabricated a predicate *<incubation_period_in_humans>* that did not exist in the KB, whereas ours correctly recalled the predicate *<minimal_incubation_period_in_humans>*, which was preserved through BGP-GNN reasoning. Using exact BGP triples is beneficial for linking the two types of fevers in SPARQL query generation.

The last two cases demonstrate that our approach can provide crucial structural information from KBs. In the third case, the BART model incorrectly considered "House of Oldenburg" as an attribute of an entity, resulting in the incorrect query fragment *?pv <pred:value>"House of Oldenburg"*, whereas our approach recognized "House of Oldenburg" as an independent entity, which was represented as *?e_1 <pred:name>"House of Oldenburg"*. Our approach then appropriately explored the one-hop neighbors in the parsed BGP graph, which included the predicate *<name_after>* underneath the question. This demonstrates the ability of our approach to identify and utilize structural information in BGP graphs. In the fourth case, the question asked about the number of independent record labels headquartered in the city where the movie "X-Men: First Class" was set in 1944. This question required incorporating quadruples from the KB, whose SPARQL query contained BLANK NODE and the keyword "Count", adding complexity to the query generation. The BART model only considered "the year 1944" as the movie's release year without considering the relationship between the movie and cities, whereas our approach utilized the quadruple connections between the cities and "X-Men: First Class". The BGP triples associated with the variable *"?r_r_tmp_0"* expressed the semantics and provided hints for the "[...]" in the SPARQL query, and our approach recovered these BGP triples to the "BLANK NODE" in the SPARQL query.

The above cases represent typical examples of how our approach improves the quality of generated SPARQL queries. In these cases, the reasoning the quality knowledge from the KB directly affects the final SPARQL generation. Moreover, our approach enhances quality by accurately matching entities and predicates, leveraging structural information from the BGP graph, and preserving semantic consistency from BGP triples to SPARQL queries.

6.7. Error analysis

We analyzed errors by randomly selecting 150 SPARQL queries that resulted in incorrect answers, categorizing the errors into five primary types, as shown in Table 4. Each primary error type has a proportion of occurrence, and certain error types are further divided into sub-types.

The most common error was caused by the BART model we utilized in the SPARQL generator, constituting 48% of the total errors and presenting a bottleneck for TrackerQA. The BART errors were further divided into error sub-types copy error (50%), counterintuition (25%), redundancy (13.89%), variable error (9.72%), and query keyword error (1.39%). The results suggest that, even if the provided BGP triples are of good quality, the BART model can incorrectly copy or ignore them or generate SPARQL

Table 3
SPARQL query generation case study results using samples from validation dataset.

Question	Is 0000 0000 8159 1906 Charlie Laughton's ISNI?	Type	Verify	Keyword	ASK
BGP Triples	[SEP] ?r_e_0 <ISNI>?r_pv_1 <triples>?r_pv_1 <pred:value>?r_v_1 <triples>?r_e_0 <pred:name>“Charles Laughton”				
SPARQL from BART	ASK { ?e <pred:name>“Charlie Laughton” ✖ . ?e <ISNI>?pv . ?pv <pred:value>“0000 0000 8159 1906” . }	SPARQL from Ours	ASK { ?e <pred:name>“Charles Laughton” ✔ . ?e <ISNI>?pv . ?pv <pred:value>“0000 0000 8159 1906” . }		
Question	What has a shorter incubation period in humans, typhoid fever or yellow fever?	Type	Logical	Keyword	UNION
BGP Triples	[SEP] ?r_e_0 <pred:name>“yellow fever” <triples>?r_e_0 <minimal_incubation_period_in_humans> ?r_pv_0 <triples> ?r_e_1 <pred:name>“typhoid fever” <triples>?r_e_1 <minimal_incubation_period_in_humans> ?r_pv_3 <triples> ?r_pv_0 <pred:value>?r_v_0 <triples>?r_pv_3 <pred:value>?r_v_3				
SPARQL from BART	SELECT ?e WHERE { { ?e <pred:name>“typhoid fever” . } UNION { ?e <pred:name>“yellow fever” . } ?e <incubation_period_in_humans> ✖ ?pv . ?pv <pred:value>?v . } ORDER BY ?v LIMIT 1	SPARQL from Ours	SELECT ?e WHERE { { ?e <pred:name>“typhoid fever” . } UNION { ?e <pred:name>“yellow fever” . } ?e <minimal_incubation_period_in_humans> ✔ ?pv . ?pv <pred:value>?v . } ORDER BY ?v LIMIT 1		
Question	How many estates in land originated in the name of the House of Oldenburg or were founded after 1070?	Type	Count	Keyword	COUNT & UNION
BGP Triples	[SEP] ?c_0 <pred:name>“estate in land” <triples>?r_e_0 <pred:name>“House of Oldenburg” <triples> ?r_e_0 <named_after>?v_e_0 <triples>?v_e_0 <inception>?v_pv_0 <triples>?v_pv_0 <pred:year>?v_v_0 <triples> ?v_e_0 <pred:instance_of>?c_0				
SPARQL from BART	SELECT (COUNT(DISTINCT ?e) AS ?count) WHERE { ?e <pred:instance_of>?c . ?c <pred:name>“estate in land” . { ?e <name>?pv . ✖ ?pv <pred:value>“House of Oldenburg” . ✖ } UNION { ?e <inception>?pv . ?pv <pred:year>?v . FILTER (?v >1070) . } }	SPARQL from Ours	SELECT (COUNT(DISTINCT ?e) AS ?count) WHERE { ?e <pred:instance_of>?c . ?c <pred:name>“estate in land” . { ?e_1 <named_after>?e . ✔ ?e_1 <pred:name>“House of Oldenburg” . ✔ } UNION { ?e <inception>?pv . ?pv <pred:year>?v . FILTER (?v >1070) . } }		
Question	The city where X-Men: First Class is set in the year 1944 is HQ of how many independent record labels?	Type	Qualifier	Keyword	[BLANK NODE] & COUNT
BGP Triples	[SEP] ?v_e_2 <headquarters_location>?v_e_0 <triples>?r_r_tmp_0 <pred:fact_t>?v_e_0 <triples> ?r_e_0 <pred:name>“X-Men: First Class” <triples>?r_r_tmp_relation_qpv_0 <pred:year>“1944” <triples> ?r_e_0 <narrative_location>?v_e_0 <triples>?v_e_0 <pred:instance_of>?c_0 <triples> ?r_r_tmp_0 <point_in_time>?r_r_tmp_relation_qpv_0 <triples>?r_r_tmp_0 <pred:fact_r>“narrative location” <triples> ?v_e_2 <pred:instance_of>?c_1 <triples>?c_1 <pred:name>“independent record label” <triples> ?r_r_tmp_0 <pred:fact_h>?r_e_0 <triples>?c_0 <pred:name>“city”				
SPARQL from BART	SELECT (COUNT(DISTINCT ?e) AS ?count) WHERE { ?e <pred:instance_of>?c . ?c <pred:name>“independent record label” . ?e <headquarters_location>?e_1 . ?e_1 <pred:instance_of>?c_1 . ?c_1 <pred:name>“city” . ?e_2 <narrative_location>?e_1 . ?e_2 <pred:name>“X-Men: First Class” . ?e_2 <point_in_time>?pv . ?pv <pred:year>1944 . ✖ }	SPARQL from Ours	SELECT (COUNT(DISTINCT ?e) AS ?count) WHERE { ?c <pred:name>“independent record label” . ?e <pred:instance_of>?c . ?c_1 <pred:name>“city” . ?e_1 <pred:instance_of>?c_1 . ?e_2 <pred:name>“X-Men: First Class” . ?e <headquarters_location>?e_1 . ?e_2 <narrative_location>?e_1 . [<pred:fact_h>?e_2 ; <pred:fact_r><narrative_location>; <pred:fact_t>?e_1] <point_in_time>?qpv . ?qpv <pred:year>1944 . ✔ }		

1. Unfaithful and incorrect SPARQL queries presented in red with the symbol “✖”; accurate queries marked in blue with the symbol “✔”. BGP triples, with the triples that contribute to generating the query are presented in blue.
2. To enhance readability, variables in the BGP triples are underscored (“_”). “?r_r_tmp_N” represent the “tmp” variables connected to the recalled entities; “?r_r_tmp_relation_qpv_N” denote the “qpv” variables associated with the recalled entities for their primary triples.
3. “Type” means the question category corresponding with the dataset description. “Keyword” indicates the query keyword in the SPARQL language syntax.

queries with redundant words and incorrect query keywords. Furthermore, in certain cases, the order of the object and subject variables in SPARQL queries was swapped. Subsequently, we observed that errors in recalling items and predicates from the KB can lead to interacting questions with unfaithful facts from the base. Furthermore, reasoning errors suggest that the interacting quality affects SPARQL query generation. Dataset annotation errors accounted for 7.33% of the total errors, potentially because of incorrect annotations, such as incorrect SPARQL query keywords. The least frequent error (2.67%) arose from the BGP parser.

Table 4
Distribution of main error types and their respective sub-error types.

Primary error type	Proportion	Sub-error type	Proportion	Description
SPARQL generator	48%	Copy	50%	The BGP triples are in good quality, but BART copies them incorrectly.
		Counterintuitive	25%	The BGP triples are reasonable, but BART has ignored them in generations.
		Redundancy	13.89%	The SPARQL queries from BART have extra words not in BGP triples.
		Variables	9.72%	The object and subject variables in SPARQL queries are swapped in order.
		Query keyword	1.39%	The SPARQL query form keywords are generated incorrectly.
Search space reducer	33.33%	Item	56%	Incorrect items are retrieved, resulting in some errors in BGP graphs.
		Predicate	44%	Incorrect predicates are retrieved, resulting in some errors in BGP graphs.
BGP-GNN reasoning	8.67%	–	–	The BGP triples output from BGP-GNN are unreasonable.
Dataset annotation	7.33%	–	–	The dataset samples have been annotated incorrectly during their creation.
BGP graph parsing	2.67%	–	–	The gold BGP triples are not labeled during the parsing of the BGP graphs.

Table 5
Accuracy scores for recalled items and predicates.

Models	Recalled items (Hit@5)	Recalled predicates (Hit@20)
RoBERTa	94.22	98.94
RoBERTa*	96.03	99.10
Δ	+1.81	+0.16
ELQ	45.87 (Micro)	–
	27.00 (Macro)	–

1. Performance of ensemble model (denoted by “*”) was obtained by aggregating the results of three individual models through majority voting. “ Δ ” represents improvements.
2. In the context of the ELQ toolkit, “Micro” refers to the overall recall accuracy rate of all items that need to be identified within the dataset. “Macro” signifies the evaluation of each sample in the dataset, thereby correctly recalling all items in each question.

In conclusion, the BART model in the SPARQL generator is the primary source of error. Therefore, addressing its limitations can significantly enhance the performance of the model. Additionally, although reducer, reasoning, dataset, and parsing errors are relatively uncommon, eliminating these can positively impact framework performance.

7. Analysis of key components in framework

In this section, we analyze the performance of components using self-supervision, including the search space reducer, parsed BGP graphs, and BGP-GNN. By extracting supervision from gold SPARQL queries, these components have demonstrated the effectiveness of self-supervision, eliminating the expensive labor for labeling.

7.1. Analysis of results for search space reduction

We assessed the retrieval accuracy scores for both items and predicates to evaluate the performance of the search space reducer. As shown in Table 5, the accuracy score for item retrieval was 94.22%. As for the ensemble model, the accuracy score increased by 1.81% to 96.03%. The accuracy scores for predicate retrieval and the ensemble model were 98.94 and 99.1%, respectively. The results indicate that the search space reducer is effective at supplying useful elements to the BGP parser. However, the performance of the out-of-the-box ELQ toolkit for item retrieval was limited, yielding accuracy scores of 45.87% (Micro) and 27% (Macro). The toolkit cannot recall the question-related predicates. Compared to the ELQ, our method extracted the items and predicates from the SPARQL queries, which are precise self-supervised training signals in training stages.

7.2. Analysis of quality of the parsed BGP graph

We evaluated the quality and sizes of the parsed BGP graphs, with the results presented in Table 6. The nodes in the graph fall into three categories: (1) items inherited from the KB, (2) variables generated during the parsing process, and (3) predicates downgraded from quadruples. The edges are of two types: (1) predicates derived from the KB, and (2) RDF schema elements, such as `<pred:name>` and `<pred:fact_h>`. To assess the quality of the graphs, we measured the coverage of the gold SPARQL triples achieved by the BGP graphs, i.e., the number of triples from the SPARQL queries encompassed by the BGP graphs. Triples were the self-supervised signals extracted from SPARQL queries using regular expressions, resulting in 592,144 and 74,411 triples in the training and validation sets to label the parsed BGP graphs. The test dataset did not include the gold SPARQL queries. We achieved correct labeling rates of 96.49 and 95.15% for the triples in the BGP graphs on the training and validation datasets, respectively. Approximately 5% of the triples were incorrectly labeled, possibly owing to item and predicate retrieval errors. However, when the ELQ toolkit was used, the results were less satisfactory, as the toolkit could label only 43.69 and 43.21% of the triples correctly in the training and validation datasets, respectively. Higher coverage provides stronger self-supervision signals for training the BGP-GNN.

Table 6

Sizes of BGP graphs constructed through our search space reducer and ELQ toolkit.

Dataset			Train	Valid	Test	Dataset			Train	Valid	Test
Ours	triples	total	592,144	74,411	–	ELQ	triples	total	592,144	74,411	–
		labeled	571,346	70,799	–			labeled	258,729	32,150	–
		coverage	96.49%	95.15%	–			coverage	43.69%	43.21%	–
	node	avg	81.53	81.78	81.76		node	avg	50.49	51.68	52.37
		max	624	574	526			max	739	694	582
		min	13	14	16			min	0	0	0
	edge	avg	217.42	216.01	218.45		edge	avg	66.54	68.27	69.18
		max	1394	1198	1268			max	1203	1085	958
		min	39	41	48			min	0	0	0

1. Terms “triples”, “labeled”, and “coverage” refer to the number of triples extracted from the SPARQL queries, the total number of triples included in the BGP graphs, and the percentage of triples that are labeled, respectively. Bolded numbers represent better performance in comparing the ELQ toolkit and our method.

2. Term “node” denotes the number of nodes contained within the BGP graphs, with the average (“avg”), maximum (“max”), and minimum (“min”) values presented. Similarly, “edge” signifies the number of edges in the BGP graphs, which can also be considered the quantity of BGP triples contained within the parsed graph.

Table 7 F_1 and accuracy scores of the BGP-GNN results.

Models	Node		Edge	
	F_1	Acc (%)	F_1	Acc (%)
BGP-GNN	0.96	99.27	0.93	99.54
BGP-GNN*	0.98	98.91	0.99	99.37
Δ	+0.02	−0.36	+0.06	−0.17
BGP-GNN (ELQ)	0.54	88.96	0.63	93.88

1. Ensemble Performance (denoted by “*”) was majority voted by three single models. “ Δ ” represents improvements.

7.3. Analysis of results for BGP-GNN reasoning

We evaluated the node and edge classification results to assess our BGP-GNN’s reasoning performance. The F_1 and accuracy scores are presented in Table 7. The performance of the BGP-GNN single model was satisfactory, with F_1 scores of 0.96 and 0.93 and accuracies of 99.27 and 99.54% for node classification and edge classification, respectively. Introducing an ensemble model increased the F_1 score by 0.02 and 0.06 for node and edge classification, respectively, but reduced the respective accuracies by 0.36 and 0.17%. This indicates that high-quality BGP triples can be obtained from the parsed BGP graphs for SPARQL query generation.

Additionally, we analyzed the classification results of BGP graphs constructed using the ELQ. The F_1 scores for node and edge classification were 0.54 and 0.63, respectively, with respective accuracies of 88.96 and 93.88%. These results indicate a significant decline in performance compared with the BGP graphs constructed via contrastive learning. We consider the less semantic information in these graphs can negatively affect the BGP-GNN reasoning.

8. Discussion

In this section, we discuss the theoretical and practical implications of our study. We also describe the limitations and how it can be extended.

8.1. Theoretical and practical implications

Theoretically, the proposed self-supervised TrackerQA framework effectively reasons out exact KB-BGP paths for accurate SPARQL query generation, significantly improving complex KBQA performance. The **search space reducer** employs two contrastive learning semantic matching models to retrieve question-related knowledge to establish a reduced space, eliminating redundant knowledge that declines the effectiveness and efficiency in path reasoning and SPARQL generation. We explore the utilization of **BGP graph** as an intermediate representation to interact questions with the KB, which enhances the representation capabilities of intermediate representation, including converting RDF triples and quadruples into BGP triples, representing question uncertainty using variables. BGP graphs not only simplify the representation of complex KBs in graph computation techniques such as GNN but also greatly improve the performance of SPARQL generation due to the BGP representation of the reasoned knowledge. In addition, we bypass the expensive and labor-intensive labeling in reasoning paths by extracting self-supervised signals from gold SPARQL queries. The proposed self-supervised **BGP-GNN** efficiently reasons out paths from the knowledge representation learning perspective, benefiting from the knowledge semantic encoding of BGP graph through a PLM, the semantic modeling of nodes and directed edges in graph transformer layers, and the question-aware bidirectional attention mechanism ultimately reasons out exact paths by classifying nodes and edges. The graph transformer layers also enable us to model dynamic, large-scale, and directed edges,

i.e., directed binary predicates in the KB, for representation learning of the BGP triples. Additionally, with the emergence of the LLMs recently, although they show promising results in various tasks, including KBQA, our work still holds significant importance for grounding in KBs. For instance, a Retrieve-then-Generate LLM-incorporated method (Luo et al., 2024) demonstrates that erroneous retrieval results and increased token counts can make training LLMs more challenging. This highlights reasoning capabilities of our approach in reducing incorrect retrievals, which not only provides precise knowledge facts but also offers structural guidance through BGP triples for the generating logical form skeletons, thereby aiding LLMs in ground within KBs.

In practical terms, we suggest utilizing contrastive learning in the KB to reduce the space for more effective and efficient reasoning. We also recommend paying more attention to the BGP in SPARQL-style complex KBQA because obtaining precise BGP reasoning paths can enhance the faithfulness and accuracy of answers interpretably. In addition to knowledge, the rest of the SPARQL query, such as query keywords and constraints, can be easily generated by generative PLM translations or even prompting of LLMs. Additionally, the BGP-GNN can facilitate the representation learning for KB, which can handle multiple and directed predicates. Besides, the introduced bidirectional attention mechanisms empower the BGP-GNN with effective reasoning capability in KBQA tasks.

8.2. Limitations and extensions

In this article, we focus on retrieving accurate facts from the RDF KB, particularly complex questions that involve multiple triples and quadruples. Our approach follows the decomposition of the Wikidata schema. However, it has limitations in accommodating all KB schema designs. Our future work will propose alternative BGP parsers to support more KB schemas in increasingly challenging KBQA questions to extend the capabilities of our approach.

Additionally, we rely on a BART model to transform the retrieved facts into SPARQL queries along with their corresponding questions. However, our experiments described in Section 6.4 show that the BART model is the bottleneck in the framework's performance. We can further enhance the capability of the BART model or explore whether LLMs can improve the performance of SPARQL query generation.

9. Conclusion

This study presents a self-supervised BGP reasoning framework to enhance SPARQL generation performance through BGP-GNN reasoning on BGP graphs in complex KBQA. We introduce contrastive learning semantic matching models to reduce the search space by recalling question-related items and predicates. We build a BGP parser to parse questions into BGP graphs, and we naturally obtain the self-supervision signals from gold SPARQL queries. Subsequently, we designed the BGP-GNN to reason out the exact paths for SPARQL generation with a BART model. Experimental results demonstrated that our single and ensemble models achieve accuracy scores of 92.40 and 95.32%, respectively. The BGP-GNN performs well in path deduction with F_1 scores of 0.98 and 0.99 for nodes and edges, respectively. This study highlights the importance of accurately reasoning knowledge paths corresponding to questions and demonstrates the effectiveness of BGP-GNN in fine-grained, exact path reasoning for question answering.

A more powerful SPARQL generator is warranted in the future. To eliminate instability during SPARQL generation, we need to integrate multiple SPARQL queries for one question. Additionally, we aim to apply BGP-GNNs to other research fields, such as knowledge graph embeddings, and explore the zero-shot generation capabilities of the LLMs provided with our accurate BGP triples.

CRedit authorship contribution statement

Feng Gao: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Yan Yang:** Writing – review & editing, Funding acquisition, Formal analysis, Conceptualization. **Peng Gao:** Writing – review & editing, Methodology, Formal analysis, Conceptualization. **Ming Gu:** Writing – review & editing, Investigation. **Shangqing Zhao:** Methodology. **Yuefeng Chen:** Validation, Resources. **Hao Yuan:** Validation, Resources. **Man Lan:** Writing – review & editing, Validation, Supervision, Funding acquisition. **Aimin Zhou:** Supervision, Methodology, Conceptualization. **Liang He:** Supervision, Project administration, Conceptualization.

Data availability

Data will be made available on request.

References

- Bai, X., Chen, Y., & Zhang, Y. (2022). Graph pre-training for AMR parsing and generation. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 6001–6015). <http://dx.doi.org/10.18653/v1/2022.acl-long.415>.
- Bi, X., Nie, H., Zhang, G., Hu, L., Ma, Y., Zhao, X., et al. (2023). Boosting question answering over knowledge graph with reward integration and policy evaluation under weak supervision. *Information Processing & Management*, 60(2), Article 103242. <http://dx.doi.org/10.1016/j.ipm.2022.103242>.
- Cao, S., Shi, J., Pan, L., Nie, L., Xiang, Y., Hou, L., et al. (2022). KQA Pro: A dataset with explicit compositional programs for complex question answering over knowledge base. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 6101–6119). <http://dx.doi.org/10.18653/v1/2022.acl-long.422>.
- Cao, S., Shi, J., Yao, Z., Lv, X., Yu, J., Hou, L., et al. (2022). Program transfer for answering complex questions over knowledge bases. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 8128–8140). <http://dx.doi.org/10.18653/v1/2022.acl-long.559>.

- Chen, Z.-Y., Chang, C.-H., Chen, Y.-P., Nayak, J., & Ku, L.-W. (2019). UHop: An unrestricted-hop relation extraction framework for knowledge-based question answering. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)* (pp. 345–356). <http://dx.doi.org/10.18653/v1/N19-1031>.
- Chen, P., Wang, Y., Yu, X., & Feng, R. (2022). QLogicE: Quantum logic empowered embedding for knowledge graph completion. *Knowledge-Based Systems*, 239, Article 107963.
- Das, R., Godbole, A., Naik, A., Tower, E., Zaheer, M., Hajishirzi, H., et al. (2022). Knowledge base question answering by case-based reasoning over subgraphs. Vol. 162, In *Proceedings of the 39th international conference on machine learning* (pp. 4777–4793).
- Das, R., Zaheer, M., Thai, D., Godbole, A., Perez, E., Lee, J. Y., et al. (2021). Case-based reasoning for natural language queries over knowledge bases. In *Proceedings of the 2021 conference on empirical methods in natural language processing* (pp. 9594–9611). <http://dx.doi.org/10.18653/v1/2021.emnlp-main.755>.
- Dong, L., & Lapata, M. (2016). Language to logical form with neural attention. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 33–43). <http://dx.doi.org/10.18653/v1/P16-1004>.
- Du, H., Huang, Q., Zhang, C., & Zhao, D. (2022). Knowledge-enhanced iterative instruction generation and reasoning for knowledge base question answering. In *Natural language processing and Chinese computing* (pp. 431–444).
- Ferré, S. (2013). SQUALL2SPARQL: A translator from controlled english to full SPARQL 1.1. Vol. 1179, In *Working notes for conference and labs of the evaluation forum*.
- Gu, Y., Deng, X., & Su, Y. (2023). Don't generate, discriminate: A proposal for grounding language models to real-world environments. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 4928–4949). Toronto, Canada: Association for Computational Linguistics, <http://dx.doi.org/10.18653/v1/2023.acl-long.270>, Retrieved from <https://aclanthology.org/2023.acl-long.270>.
- Gu, Y., & Su, Y. (2022). ArcaneQA: Dynamic program induction and contextualized encoding for knowledge base question answering. In *Proceedings of the 29th international conference on computational linguistics* (pp. 1718–1731).
- Han, J., Cheng, B., & Wang, X. (2020). Two-phase hypergraph based reasoning with dynamic relations for multi-hop KBQA. In *Proceedings of the 29th international joint conference on artificial intelligence* (pp. 3615–3621). <http://dx.doi.org/10.24963/ijcai.2020/500>.
- He, G., Lan, Y., Jiang, J., Zhao, W. X., & Wen, J.-R. (2021). Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM international conference on web search and data mining* (pp. 553–561). <http://dx.doi.org/10.1145/3437963.3441753>.
- Herzig, J., Shaw, P., Chang, M., Guu, K., Pasupat, P., & Zhang, Y. (2021). Unlocking compositional generalization in pre-trained models using intermediate representations. CoRR, abs/2104.07478.
- Hu, X., Wu, X., Shu, Y., & Qu, Y. (2022). Logical form generation via multi-task learning for complex question answering over knowledge bases. In *Proceedings of the 29th international conference on computational linguistics* (pp. 1687–1696).
- Jiao, J., Wang, S., Zhang, X., Wang, L., Feng, Z., & Wang, J. (2021). gMatch: Knowledge base question answering via semantic matching. *Knowledge-Based Systems*, 228, Article 107270.
- Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <http://dx.doi.org/10.1109/TBDDATA.2019.2921572>.
- Kapanipathi, P., Abdelaziz, I., Ravishankar, S., Roukos, S., Gray, A., Fernandez Astudillo, R., et al. (2021). Leveraging abstract meaning representation for knowledge base question answering. In *Findings of the association for computational linguistics: ACL-IJCNLP 2021* (pp. 3884–3894). <http://dx.doi.org/10.18653/v1/2021.findings-acl.339>.
- Lan, Y., He, G., Jiang, J., Jiang, J., Zhao, W. X., & Wen, J.-R. (2023). Complex knowledge base question answering: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(11), 11196–11215. <http://dx.doi.org/10.1109/TKDE.2022.3223858>.
- Lan, Y., Wang, S., & Jiang, J. (2019). Knowledge base question answering with a matching-aggregation model and question-specific contextual relations. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 27(10), 1629–1638. <http://dx.doi.org/10.1109/TASLP.2019.2926125>.
- Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., et al. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 7871–7880). <http://dx.doi.org/10.18653/v1/2020.acl-main.703>.
- Li, Z., Fan, S., Gu, Y., Li, X., Duan, Z., Dong, B., et al. (2024). FlexKBQA: A flexible LLM-powered framework for few-shot knowledge base question answering. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17), 18608–18616. <http://dx.doi.org/10.1609/aaai.v38i17.29823>, Retrieved from <https://ojs.aaai.org/index.php/AAAI/article/view/29823>.
- Li, B. Z., Min, S., Iyer, S., Mehdad, Y., & Yih, W.-t. (2020). Efficient one-pass end-to-end entity linking for questions. In *Proceedings of the 2020 conference on empirical methods in natural language processing EMNLP*, (pp. 6433–6441). <http://dx.doi.org/10.18653/v1/2020.emnlp-main.522>.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach. CoRR, abs/1907.11692.
- Liu, Y., Yavuz, S., Meng, R., Radev, D., Xiong, C., & Zhou, Y. (2022). Uni-Parser: Unified semantic parser for question answering on knowledge base and database. In *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 8858–8869). <http://dx.doi.org/10.18653/v1/2022.emnlp-main.605>.
- Lu, J., Tan, X., Pergola, G., Gui, L., & He, Y. (2022). Event-centric question answering via contrastive learning and invertible event transformation. In *Findings of the association for computational linguistics: EMNLP 2022* (pp. 2377–2389).
- Luo, H., E. H., Tang, Z., Peng, S., Guo, Y., Zhang, W., et al. (2024). ChatKBQA: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. Retrieved from <https://openreview.net/forum?id=tqhAA26vXE>.
- Luo, L., Fang, Y., Lu, M., Cao, X., Zhang, X., & Zhang, W. (2023). GSim: A graph neural network based relevance measure for heterogeneous graphs. *IEEE Transactions on Knowledge and Data Engineering*, 1–14. <http://dx.doi.org/10.1109/TKDE.2023.3271425>.
- Miller, A., Fisch, A., Dodge, J., Karimi, A.-H., Bordes, A., & Weston, J. (2016). Key-value memory networks for directly reading documents. In *Proceedings of the 2016 conference on empirical methods in natural language processing* (pp. 1400–1409). <http://dx.doi.org/10.18653/v1/D16-1147>.
- Myung, J., Yeon, J., & Lee, S.-g. (2010). SPARQL basic graph pattern processing with iterative MapReduce. In *Proceedings of the 2010 workshop on massive data analytics on the cloud*. <http://dx.doi.org/10.1145/1779599.1779605>.
- Nie, L., Cao, S., Shi, J., Sun, J., Tian, Q., Hou, L., et al. (2022). Graphq IR: Unifying the semantic parsing of graph query languages with one intermediate representation. In *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 5848–5865). <http://dx.doi.org/10.18653/v1/2022.emnlp-main.394>.
- Qiu, Y., Wang, Y., Jin, X., & Zhang, K. (2020). Stepwise reasoning for multi-relation question answering over knowledge graph with weak supervision. In *Proceedings of the 13th international conference on web search and data mining WSDM '20*, (pp. 474–482). <http://dx.doi.org/10.1145/3336191.3371812>.
- Saint-Dizier, P., & Moens, M.-F. (2011). Knowledge and reasoning for question answering: Research perspectives. *Information Processing & Management*, 47(6), 899–906.
- Saxena, A., Tripathi, A., & Talukdar, P. (2020). Improving multi-hop question answering over knowledge graphs using knowledge base embeddings. In *Proceedings of the 58th annual meeting of the association for computational linguistics* (pp. 4498–4507). <http://dx.doi.org/10.18653/v1/2020.acl-main.412>.
- Schlichtkrull, M., Kipf, T. N., Bloem, P., van den Berg, R., Titov, I., & Welling, M. (2018). Modeling relational data with graph convolutional networks. In *The semantic web – 15th international conference, extended semantic web conference* (pp. 593–607).
- Shen, T., Geng, X., Long, G., Jiang, J., Zhang, C., & Jiang, D. (2020). Effective search of logical forms for weakly supervised knowledge-based question answering. In *Proceedings of the twenty-ninth international joint conference on artificial intelligence, IJCAI-20* (pp. 2227–2233). <http://dx.doi.org/10.24963/ijcai.2020/308>, (Main track).

- Shivashankar, K., Benmaarouf, K., & Steinmetz, N. (2022). From graph to graph: AMR to SPARQL. In *Proceedings of the 7th natural language interfaces for the web of data (nLIWoD) co-located with the 19th European semantic web conference (ESWC 2022)*.
- Shu, Y., Yu, Z., Li, Y., Karlsson, B., Ma, T., Qu, Y., et al. (2022). TIARA: Multi-grained retrieval for robust question answering over large knowledge base. In *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 8108–8121). <http://dx.doi.org/10.18653/v1/2022.emnlp-main.555>.
- Sun, Y., Zhang, L., Cheng, G., & Qu, Y. (2020). SPARQA: Skeleton-based semantic parsing for complex questions over knowledge bases. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05), 8952–8959. <http://dx.doi.org/10.1609/aaai.v34i05.6426>.
- Tan, Y., Min, D., Li, Y., Li, W., Hu, N., Chen, Y., et al. (2023). Can ChatGPT replace traditional KBQA models? An in-depth analysis of the question answering performance of the GPT llm family. In *Proceedings of the 22nd international semantic web conference, ISWC 2023*.
- Tran, P. N., & Nguyen, D. T. (2016). A linked data driven semantic model for interpreting english queries in question answering system. In *Proceedings of the 10th international conference on ubiquitous information management and communication*. <http://dx.doi.org/10.1145/2857546.2857568>.
- Wang, S., & Qin, B. (2024). A novel joint training model for knowledge base question answering. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 32, 666–679. <http://dx.doi.org/10.1109/TASLP.2023.3336526>.
- Ye, X., Yavuz, S., Hashimoto, K., Zhou, Y., & Xiong, C. (2022). RNG-KBQA: Generation augmented iterative ranking for knowledge base question answering. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 6032–6043). <http://dx.doi.org/10.18653/v1/2022.acl-long.417>.
- Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., et al. (2021). Do transformers really perform badly for graph representation? Vol. 34, In *Advances in neural information processing systems* (pp. 28877–28888).
- Zhang, Q., Chen, S., Fang, M., & Chen, X. (2023). Joint reasoning with knowledge subgraphs for multiple choice question answering. *Information Processing & Management*, 60(3), Article 103297. <http://dx.doi.org/10.1016/j.ipm.2023.103297>.
- Zhang, C., D'Haro, L. F., Zhang, Q., Friedrichs, T., & Li, H. (2023). PoE: A panel of experts for generalized automatic dialogue assessment. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31, 1234–1250.
- Zhang, G., Liu, J., Zhou, G., Xie, Z., Yu, X., & Cui, X. (2023). Query path generation via bidirectional reasoning for multihop question answering from knowledge bases. *IEEE Transactions on Cognitive and Developmental Systems*, 15(3), 1183–1195. <http://dx.doi.org/10.1109/TCDS.2022.3198272>.
- Zhang, J., Zhang, L., Hui, B., & Tian, L. (2022). Improving complex knowledge base question answering via structural information learning. *Knowledge-Based Systems*, 242, Article 108252. <http://dx.doi.org/10.1016/j.knosys.2022.108252>.
- Zhang, L., Zhang, J., Wang, Y., Cao, S., Huang, X., Li, C., et al. (2023). FC-KBQA: A fine-to-coarse composition framework for knowledge base question answering. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 1002–1017). <http://dx.doi.org/10.18653/v1/2023.acl-long.57>.